GitHub スター上位レポート（過去 365 日）

2026-10-30 作成 ／ 対象: 2025-10-30 以降に作成されたリポジトリ
テーマ: RAG、GraphRAG、reranker、context engineering、prompt engineering、LLM fine-tuning、LoRA LLM、RLHF、GRPO、LLM distillation、test-time scaling LLM、agent memory

#	区分	リポジトリ	★	一言で
1	テーマ	affaan-m/ECC	292,954	Claude CodeやCodexなどのAIコーディングエージェントに、計画・テスト・レビュー・記憶の仕組みを後付けするハーネス最適化システム
2	テーマ	Graphify-Labs/graphify	139,010	コードやドキュメント、PDF などをまとめて、AI コーディングアシスタントから問い合わせできるナレッジグラフに変換する CLI ツール兼スキルです。
3	テーマ	headroomlabs-ai/headroom	78,312	AIエージェントがLLMに渡すツール出力・ログ・ファイル・RAGチャンクをローカルで圧縮し、トークンを削減するライブラリ兼プロキシ兼MCPサーバー
4	テーマ	code-yeongyu/oh-my-openagent	70,493	プロンプトに ultrawork や mass ulw と書き足すだけで、複数モデル・多数のエージェントに調査から実装までを任せられる AI エージェントハーネス「OmO」
5	テーマ	gsd-build/get-shit-done	64,182	Claude Code向けのメタプロンプティング・コンテキストエンジニアリング・仕様駆動開発システム「GSD」の旧リポジトリ（現在はアーカイブ済み）
6	テーマ	tirth8205/code-review-graph	32,494	Tree-sitterでコードベースを知識グラフ化し、MCP経由でAIコーディングツールに必要最小限のレビュー文脈を渡すローカルファーストのツール
7	テーマ	muratcankoylan/Agent-Skills-for-Context-Engineering	18,248	AIエージェント開発に必要なコンテキストエンジニアリングの知見を、Agent Skills形式でまとめたオープンなスキル集
8	テーマ	Orchestra-Research/AI-Research-SKILLs	14,112	Claude CodeやCodex、Gemini CLIなどのAIエージェントに、AI研究の全工程を自律的に進めるための知識を与えるSkillsライブラリ
9	テーマ	adongwanai/AgentGuide	11,450	JavaGuideを手本にした、AI Agent開発の学習から大模型関連職の就職面接対策までを網羅する中国語のオープンソース知識ベース
10	テーマ	MakazhanAlpamys/Soup	9,808	YAML一つでLLMのファインチューニングから書き出しまでを行うCLIで、4GB GPUで8Bモデルを学習できるレイヤーストリーミングを備える
11	テーマ	Gen-Verse/OpenClaw-RL	5,742	日常の会話フィードバックを学習信号に変え、OpenClaw上のエージェントを完全非同期の強化学習で継続的に最適化するフレームワーク

12	テーマ	walkinglabs/hands-on-modern-rl	4,823	CartPoleからRLHF・GRPO・Agentic RLまでを一貫して学べる、実験重視のオープンな強化学習教材
13	テーマ	amitshekhariitbhu/ai-engineering-interview-questions	3,588	AIエンジニア職の面接対策として、LLM・RAG・エージェント等の頻出質問と解説記事へのリンクをまとめたチートシート集
14	テーマ	aiming-lab/MetaClaw	3,465	普段の会話から学習し進化するパーソナルAIエージェント向けのプロキシ型メタ学習フレームワーク
15	テーマ	Ch1rpys/Mirrai	1,268	実際のチャット履歴から相手の人格を再構築し、最後は「卒業」で別れまで支える自前ホスト型AIコンパニオン
16	テーマ	lasgroup/SDPO	1,137	LLMの強化学習に自己蒸留を組み合わせ、豊富なフィードバックを密な学習信号に変えるSDPOの公式実装
17	テーマ	CodeBendKit/codeseek	761	AIコーディングエージェント向けに、コールグラフとハイブリッド意味検索を提供するRust製CLI
18	テーマ	MoMoM101/RAG-ReActAgent	208	Qdrant と BM25 のハイブリッド検索、自作 ReAct ループ、引用検証を組み合わせたローカル優先の文書ナレッジベース QA システム
19	テーマ	zhengkid/AutoTTS	178	LLMのテスト時スケーリング（TTS）戦略を、コーディングエージェントがオフライン再生環境上で自動探索する研究「AutoTTS」の公式実装
20	テーマ	sunshy-1/SpecTTS-Bench	114	LLMのテスト時スケーリングを投機的デコーディングで高速化する手法を公平に比較するICLR'26論文のベンチマーク実装

テーマ 1. affaan-m/ECC

https://github.com/affaan-m/ECC

★ 292,954 ／ フォーク 43,345 ／ 言語 JavaScript ／ 作成 2026-01-18 ／ MIT
該当テーマ: agent memory

Claude CodeやCodexなどのAIコーディングエージェントに、計画・テスト・レビュー・記憶の仕組みを後付けするハーネス最適化システム

概要

ECCは、AIコーディングエージェントに「計画→テスト→実装→レビュー→検証→記憶→改善」という一連のエンジニアリング手順を組み込むためのオープンソースのツール群です。71のエージェント、314のスキル、98のコマンド、フック、ルール、メモリ、継続学習、AgentShieldによるセキュリティスキャンを一度インストールするだけで使えるようになります。主な対象はClaude Codeで、CodexとQoderにはネイティブプラグインがあり、Cursor、OpenCode、Gemini、Zed、GitHub Copilotなど多数のハーネスにも機能を限定したアダプタを用意しています。毎回プロンプトで手順を組み立て直さずに済み、「コンテキストウィンドウは最適化し、それ以外は永続化する」という方針で、エージェントの作業品質を底上げすることを狙っています。

要点

- 71エージェント・314スキル・98コマンドを一括で提供
- Claude Codeを中心に、Codex・Qoder・Cursorなど多数のハーネスに対応
- npx ecc-universal setup によるガイド付きインストール
- フック、メモリ、継続学習、AgentShieldによるセキュリティ検査を同梱
- インストール方法は1ハーネスにつき1つに絞る必要がある

何をするものか

ECCは、コードを書けるAIエージェントに「協調したエンジニアリングの仕組み」と道具箱を与えるシステムです。エージェントは作業前に計画を立て、テストで変更を検証し、新しいコンテキストで自分の成果物をレビューします。さらに重要な情報を記憶し、繰り返しうまくいったやり方を再利用できるスキルやワークフローに変えていきます。READMEでは、この手順を毎回プロンプトで組み立てるのではなく、一度インストールしてエージェントの標準的な動作にすることを主眼としています。MITライセンスのOSSで、プライベートリポジトリ向けにはホスト型のGitHub Appである「ECC Pro」(1シート月19ドルから)も別途提供されています。

主な機能

機能は次のとおりです。agentsには、planner、architect、tdd-guide、code-reviewer、security-reviewer、build-error-resolverなどの専門サブエージェントがあり、Go・Python・Rust・Java・Kotlin・C++向けのレビュー担当やビルド修復担当も含まれます。skillsには、TDD、リサーチ、セキュリティ、フロントエンド、ML運用、市場調査や投資家向け資料作成など、必要なときに読み込むワークフローが揃っています。commandsは/ecc:plan、/code-review、/build-fix、/context-budget、/save-sessionなどの入口で、スキル中心の構成へ移行するまでの互換レイヤーと位置付けられています。rulesは言語別に選べる常時読み込みの規

約です。hooksはセッションの要約や継続学習、instinct(信頼度付きで学習されたパターン)を扱い、AgentShieldはプロンプト、フック、MCP設定、権限、シークレットをスキャンします。

仕組み・アーキテクチャ

リポジトリのルートが唯一の情報源で、agents/、skills/、commands/、rules/、hooks/、scripts/に正本を置いています。.claude-plugin/、.codex/、.codex-plugin/、.cursor/、.opencode/、.qoder-plugin/、.kiro/などの各ハーネス向けディレクトリは、同じワークフローをそれぞれの形式にパッケージ化したり対応付けたりするアダプタで、別のコピーを個別に保守する方式ではありません。インストーラはinstall-stateにECCが管理するファイルを記録します。これによりdoctor、repair、uninstallを安全に実行でき、ユーザーが作ったファイルは上書きしません。MCPはchrome-devtoolsだけを既定のコネクタとし、それ以外はCLIやREST APIをラップしたスキル、または任意で追加するカタログ項目という方針です。

技術構成

GitHubの主要言語はJavaScriptで、scripts/配下のインストールや修復、フック実装はクロスプラットフォーム対応のNode.jsスクリプトで書かれています。READMEのバッジにはShell、TypeScript、Python、Go、Java、Perl、Markdownが並んでいます。エージェントやスキルの本体はMarkdown(SKILL.mdなど)で、Codex向けにはTOML、Kiro向けにはJSONの定義もあります。配布はnpmパッケージのecc-universalとecc-agentshieldで行われ、Node.js 18以上が必要です。Claude Codeのプラグインとして使う場合は、GitとClaude Code 2.1以上も必要になります。CIにはGitHub Actionsを使い、SLSA3の公開ワークフローやサプライチェーン監視も含まれています。マニフェストファイルが提示されていないため、依存ライブラリの詳細はREADMEからは不明です。

使い方・導入方法

推奨される導入方法は npx ecc-universal@2.2.3 setup によるガイド付きセットアップです。pnpm dlx、yarn dlx、bunxでも同じように実行できます。Claude Code内で /plugin marketplace add https://github.com/affaan-m/ECC と /plugin install ecc@ecc を実行するネイティブな方法もあります。Claude Code、Codex、Kimi Codeをまとめて設定したい場合は install --guided を使います。Codexでは codex plugin marketplace add affaan-m/ECC を実行します。その他のハーネスはリポジトリをcloneしてから ./install.sh --profile minimal --target cursor のようにターゲットを指定します。プラグインではrulesを配布できないため、rules/commonと使う言語のルールを手動で ~/.claude/rules/ecc/ にコピーします。使い始めは、機能開発なら /ecc:plan から tdd-workflow へ、レビューなら /code-review というように、目的に合わせて入口を選ぶよう案内されています。

注目ポイント・注意点

同じハーネスにインストール方法を重ねると、スキルやフックが重複します。READMEでは「方法は1ハーネスにつき1つ」と繰り返し強調されています。マルウェア混入のおそれがあるため、公式のGitHub、npm、GitHub App、ecc.tools以外からは入手しないよう警告もあります。フックを有効にするかどうかは明示的に選ぶ必要があり、--enable-hooksか--no-hooksを指定しないとインストーラは書き込み前に停止します。Claude Code以外のハーネスは機能が限定されるため、READMEのサポート状況表で確認することが勧められています。/multi-*系のコマンドには外部のccg-workflowが、/astra-reviewにはCodex CLIが別途必要です。自前のモデルゲートウェイやセルフホストモデルにも、各ハーネス側の設定で対応できます。

技術構成: JavaScript、Node.js、TypeScript、Shell、PowerShell、Python、Markdown、TOML、npm (ecc-universal, ecc-agentshield)、MCP (chrome-devtools)、GitHub Actions、Claude Code plugin、Codex plugin
向いている人: Claude CodeやCodex、Cursorなどで日常的にAIコーディングエージェントを使い、計画、TDD、レビュー、セキュリティ検査を標準化したい開発者やチームに向いています。エージェントの設定やスキル、フックを体系的に管理したい人、複数のハーネスで同じワークフローを使いたい人、エージェントの設定ファイルに対するセキュリティ監査を行いたい人にも適しています。

テーマ 2. Graphify-Labs/graphify

https://github.com/Graphify-Labs/graphify

★ 139,010 ／ フォーク 13,261 ／ 言語 Python ／ 作成 2026-04-03 ／ Apache-2.0
該当テーマ: RAG、GraphRAG

コードやドキュメント、PDF などをまとめて、AI コーディングアシスタントから問い合わせできるナレッジグラフに変換する CLI ツール兼スキルです。

概要

graphify は、プロジェクト内のコード・ドキュメント・SQL スキーマ・設定ファイル・PDF・画像・動画を一つのナレッジグラフにまとめます。AI アシスタントは grep でファイルを読み回す代わりに、このグラフに問い合わせて構造を把握できます。コードは tree-sitter による AST 解析でローカルかつ決定的に処理され、LLM もベクトルストアも使いません。各エッジには EXTRACTED(ソースに明示)か INFERRED(解決処理で推定)のタグが付き、根拠を確認できる点が特徴です。Claude Code、Cursor、Codex、Gemini CLI など 20 以上のアシスタントで /graphify スキルとして動作します。

要点

- tree-sitter で約40言語をローカル解析し、コード部分は LLM 不要
- エッジごとに EXTRACTED / INFERRED の信頼度タグが付く
- 埋め込みやベクトル DB を使わない、実際にたどれるグラフ
- query / path / explain コマンドで graph.json に問い合わせ可能
- Claude Code、Cursor、Codex など20以上のアシスタントに対応

何をするものか

AI コーディングアシスタントで /graphify . と入力すると、プロジェクト全体をナレッジグラフに変換します。出力先は graphify-out/ ディレクトリで、生成されるファイルは3つです。graph.html はブラウザで開いてノードのクリック、フィルタ、検索ができる可視化です。GRAPH_REPORT.md には主要概念、意外なつながり、推奨される質問などのハイライトがまとまります。graph.json はグラフ本体で、ファイルを読み直さずにいつでも問い合わせできます。グラフには graph.schema_version と graph.graphify_version が含まれるため、連携ツール側で形式の非互換を検出できます。README では、エージェントがファイルを grep して回る代わりにグラフを問い合わせることを主な狙いとしています。

主な機能

God nodes 機能は、最も多くのつながりを持つ概念を示し、処理がどこを通るかを把握しやすくします。Leiden 法によるコミュニティ検出では、グラフをサブシステム単位に分割し、LLM を使わずにラベルを付けます。calls / imports / inherits / mixes_in といったファイル横断の関係は、約40言語で解決されます。# NOTE: や # WHY: のコメント、ADR/RFC の引用もノードとして扱われ、コードとリンクされます。ほかに、arXiv 論文や YouTube 動画をグラフに追加する graphify add、Markdown wiki の生成(--wiki)、Mermaid のコールフロー HTML 出力があります。graphify prs による PR ダッシュボードとコンフリクト分析、Neo4j や FalkorDB へのプッシュにも対応しています。

仕組み・アーキテクチャ

コードは tree-sitter の AST で決定的に解析されます。ファイル一覧の graphify/extractors/ には、go、rust、csharp、sql、terraform、cobol、verilog など言語ごとの抽出器が並んでいます。ドキュメント、PDF、画像は意味解析パスで処理され、使う LLM は IDE セッションのモデルか、設定した API キーのバックエンドです。動画と音声は faster-whisper でローカルに文字起こしされます。クラスタリングには Leiden を使い、leiden extra が無い場合は networkx の Louvain に自動でフォールバックします。Claude Code や Gemini CLI ではフックが検索系ツール呼び出しの前に発火し、アシスタントをグラフ参照へ誘導します。Codex や Cursor では AGENTS.md や .cursor/rules/ などの指示ファイルで同じ誘導を行います。MCP サーバー(graphify.serve)も用意されており、Dockerfile から HTTP サービスとして起動できます。

技術構成

本体は Python 3.10 以上で書かれており、PyPI パッケージ名は graphifyy(y が2つ)です。コマンド名は graphify のままです。解析の中核は tree-sitter で、グラフ処理には networkx、Leiden 実装には graspologic(Python 3.13 未満の場合)などが使われます。オプション extra として、pdf、office、video(faster-whisper、yt-dlp)、mcp、watch、neo4j、falkordb、sql、postgres、ollama、openai、gemini、anthropic、bedrock、chinese(jieba)などが用意されています。Pi coding agent 向けのガードは TypeScript(graphify/pi/index.ts)で実装されています。Dockerfile は python:3.12-slim ベースで、非 root ユーザーで MCP サーバーを実行します。ライセンスは Apache-2.0 で、ルートには LICENSE-MIT も置かれています。

使い方・導入方法

導入は uv tool install graphifyy(または pipx install graphifyy)で CLI を入れ、続けて graphify install を実行するだけです。graphify install は使用中のアシスタントを検出してスキルを登録します。あとはアシスタント内で /graphify . と入力すればグラフが構築されます。Codex では /graphify の代わりに $graphify を使います。リポジトリ単位で導入したい場合は --project を付けます。主なコマンドは次の通りです。graphify query "質問" は質問に関係する部分グラフを返し、graphify path A B は2つの要素間の最短経路を、graphify explain X は1つの概念の説明を返します。更新関連では、--update で変更ファイルだけを再抽出でき、graphify hook install でコミット時の自動再構築を設定できます。プロジェクト固有の設定は .graphifyrc に書き、除外したいファイルは .graphifyignore で指定します。

注目ポイント・注意点

コードのみのコーパスなら API キー不要で、完全オフラインで動作します。一方、ドキュメントや PDF の意味解析では外部 LLM にデータが送られます。データの所在に制約がある場合は、GRAPHIFY_BACKEND=ollama でローカル処理に限定するか、GRAPHIFY_BACKEND=none で LLM を無効化できます。テレメトリは無いと明記されていますが、クエリ自体は ~/.cache/graphify-queries.log にローカル記録されます(GRAPHIFY_QUERY_LOG_DISABLE=1 で無効化可能)。PyPI には無関係な graphify* パッケージがあるため、インストール時はパッケージ名に注意してください。ベンチマークでは LOCOMO の recall@10 が 0.497 と示されていますが、QA 精度では supermemory を下回る項目もあります。README には有料の Graphify Cloud の宣伝も多く含まれます。

技術構成: Python、tree-sitter、networkx、graspologic (Leiden)、faster-whisper、yt-dlp、MCP (mcp, starlette, uvicorn)、Neo4j / FalkorDB、TypeScript (Pi ガード)、Docker
向いている人: Claude Code、Cursor、Codex、Gemini CLI などの AI コーディングアシスタントを日常的に使う開発者に向いています。大規模なコードベースや見慣れないリポジトリの構造を素早く把握したい人、エージェントのトークン消費を抑えたいチームにも有用です。コードを外部に出さずに解析したい組織や、PR レビューやアーキテクチャ把握にグラフを活用したい人にも適しています。

テーマ 3. headroomlabs-ai/headroom

https://github.com/headroomlabs-ai/headroom

★ 78,312 ／ フォーク 6,107 ／ 言語 Python ／ 作成 2026-01-07 ／ Apache-2.0
該当テーマ: RAG

AIエージェントがLLMに渡すツール出力・ログ・ファイル・RAGチャンクをローカルで圧縮し、トークンを削減するライブラリ兼プロキシ兼MCPサーバー

概要

Headroomは、Claude CodeやCursor、CodexなどのAIエージェントが読み込むツール出力、ログ、RAG結果、ファイル、会話履歴を、LLMに届く前に圧縮するコンテキスト圧縮レイヤーです。READMEでは、コーディングエージェントで約20%、JSONでは60〜95%のトークン削減をうたい、回答精度は維持されるとしています。圧縮はすべて手元のマシンで行われ、元データはローカルにキャッシュされるため、必要なときに取り出せる可逆性（CCR）を備えています。Pythonライブラリ、ゼロコード変更のプロキシ、エージェントを1コマンドで包むwrap、MCPサーバーという複数の導入形態を持つのが特徴です。

要点

- JSON・コード・テキストを種類別の圧縮器で処理しトークン削減
- 圧縮はローカル実行で、原文はCCRで後から取得可能
- headroom wrap claude などでエージェントを1コマンドで対応
- プロキシ、Python/TS SDK、MCPサーバーなど多様な導入方法
- コア処理はRust実装で、圧縮遅延は1ミリ秒未満とされる

何をするものか

Headroomは、AIエージェントやLLMアプリがプロバイダ（Anthropic、OpenAI、Bedrockなど）に送るプロンプトの手前に入り、ツール出力やログ、RAGチャンク、ファイル、会話履歴を圧縮する仕組みです。長いエージェントセッションでは同じようなJSON配列やログ行が大量にコンテキストを占めがちで、それがコストとコンテキストウィンドウを圧迫します。READMEのデモでは10,144トークンのログダンプを1,260トークンに縮めつつ、重要なFATAL行はそのまま残る様子が示されています。また入力だけでなく、モデルが書き返す出力トークンを抑える Output token reduction 機能も備えています。

主な機能

Pythonやタイプスクリプトから呼べる compress(messages) ライブラリ、headroom proxy --port 8787 で起動するドロップイン型プロキシ、Claude Code・Codex・Aider・Cline・Gooseなど多数のエージェントを包む headroom wrap、headroom_compress・headroom_retrieve・headroom_stats を提供するMCPサーバーがあります。さらに、Claude・Codex・Gemini・Grokで共有されるクロスエージェントメモリ（自動重複排除付き）、失敗セッションを解析して CLAUDE.local.md や AGENTS.md に修正点を書き込む headroom learn、簡潔な応答を促す出力整形（HEADROOM_OUTPUT_SHAPER=1 で有効化、デフォルトは無効）なども含まれます。

仕組み・アーキテクチャ

リクエストはCacheAligner、ContentRouter、CCRの順に処理されます。ContentRouterが内容の種類を判定し、JSONにはSmartCrusher、ソースコードにはAST対応のCodeCompressor、文章にはHuggingFace上の独自モデルKompress-v2-baseを適用します。SmartCrusherはキーワードリストではなくフィールドの分散統計に基づき、エラー項目や外れ値、先頭・末尾を残します。CacheAlignerはプロバイダのKVキャッシュを壊す変動要素を検出しますが、プロンプトは書き換えません。Live-zone compressionにより新しく追加された部分だけを圧縮し、凍結済みのプレフィックスはバイト単位で同一に保つため、プロンプトキャッシュが維持されます。CCRは原文をローカルに保存し、モデルが必要時に取り出せるようにします。

技術構成

主要言語はPythonですが、Cargoワークスペースとして headroom-core、headroom-proxy、headroom-pyなどのRustクレートを持ち、圧縮処理の中核やプロキシはRustで実装されています。Python拡張はpyo3とmaturinでビルドされます。プロキシ側はaxum、tokio、reqwest（rustls）を使い、Bedrock向けにaws-sigv4やaws-configも利用しています。ONNX RuntimeによるMagikaのコンテンツ検出や埋め込み関連度、BM25関連度、tiktokenやHFトークナイザ、CCRのバックエンドとしてインメモリ・SQLite・Redisがファイル構成から確認できます。TypeScript SDKはnpmで提供されます。

使い方・導入方法

uv tool install --python 3.13 "headroom-ai[all]" または pip install "headroom-ai[all]" でCLI込みのPython版を導入でき、Python 3.11以上が必要です。npm install headroom-ai はTypeScript SDKのみでCLIは含まれません。Dockerイメージ ghcr.io/headroomlabs-ai/headroom:latest も用意されています。導入後は headroom deploy、headroom wrap claude、headroom proxy のいずれかを選び、headroom doctor で動作確認、headroom dashboard で削減量を確認します。LangChain、LiteLLM、Vercel AI SDK、Agno、ASGIミドルウェアなど向けのアダプタもあり、[langchain] などの追加extrasで個別に導入します。

注目ポイント・注意点

READMEのベンチマークでは、コード検索で21%、SREインシデント調査で57%の削減を示し、GSM8Kなどの精度はベースラインと同等とされています。一方で、削減効果はデータの反復性に依存し、短い会話や散文、すでに密な出力ではほとんど縮まらないと明記されています。匿名テレメトリはデフォルトで有効で、HEADROOM_TELEMETRY=off や DO_NOT_TRACK=1 で無効化できます。x86環境ではAVX2がないとONNX機能が代替経路にフォールバックし、Intel macOSではONNX Runtimeの手動指定が必要な場合があります。企業向けの有償版Headroom Enterpriseも案内されていますが、本リポジトリはApache-2.0のオープンソースです。

技術構成: Python、Rust、TypeScript、pyo3、maturin、axum、tokio、reqwest、ONNX Runtime、Magika、HuggingFace (Kompress-v2-base)、FastAPI、MCP、SQLite、Redis、Docker
向いている人: Claude Code、Codex、Cursorなどのコーディングエージェントを日常的に使い、ツール出力の多い長時間セッションでトークンコストやコンテキスト消費を抑えたい開発者に向いています。複数エージェント間でメモリを共有したい人、既存コードを変えずにプロキシで圧縮を挟みたいチーム、RAGやLLMアプリにローカルで可逆な圧縮処理を組み込みたいエンジニアにも適しています。

テーマ 4. code-yeongyu/oh-my-openagent

https://github.com/code-yeongyu/oh-my-openagent

★ 70,493 ／ フォーク 5,819 ／ 言語 TypeScript ／ 作成 2025-12-03 ／ NOASSERTION
該当テーマ: prompt engineering

プロンプトに ultrawork や mass ulw と書き足すだけで、複数モデル・多数のエージェントに調査から実装までを任せられる AI エージェントハーネス「OmO」

概要

OmO は、ターミナルで omo コマンドを実行し、やりたい仕事を説明するだけで、調査、資料作成、バックエンドやフロントエンドのコード実装までをエージェントが進めるツールです。pi のフォークである senpi の上で動き、ultrawork キーワードによる計画と検証付きの作業、mass ulw によるエージェントのグラフ並列実行、Kibitzer と呼ばれる記憶システムを備えています。Claude、ChatGPT、Kimi、GLM のサブスクリプションでログインでき、タスクごとに適したモデルへ自動で振り分けられます。エージェントを途中で人間が助けなくても最後まで仕上がることを目指しており、AI コーディングエージェントを本格的に使い込みたい開発者に向いています。

要点

- ultrawork（ulw）を付けると、読解・計画・検証を経て完了まで作業する
- mass ulw で仕事をエージェントのグラフに分解し、並列に実行する
- git 管理の Markdown を記憶に使い、小型モデルが過去の失敗を踏まえて助言する
- 複数プロバイダのモデルを混在させ、モデルごとに調整済みのプロンプトで動く
- OpenCode 版や LazyCodex からは omo setup で設定を移行できる

何をするものか

OmO は、一つの omo コマンドでトークンを完成した成果物に変えることをうたう AI エージェントツールです。README には、一万件規模の情報源にまたがる調査、伝わるスライド資料、バックエンド、フロントエンド、コードといった用途が挙げられています。基盤は pi（badlogic/pi-mono）のフォークである senpi です。作者は、人間がエージェントからハンドルを取り戻さなければならない状況をシステムの失敗と捉えています。依頼された内容をエージェントが自分で調べ、計画し、実装し、検証し、終わるまで続けることを設計思想としています。リポジトリ名は oh-my-openagent ですが、以前は OpenCode 向けのプラグイン（oh-my-opencode）として提供されていた経緯が、ファイル構成や移行ガイドからうかがえます。

主な機能

中心となる機能はキーワードによる作業モードの切り替えです。プロンプトに ultrawork（ulw）を加えると、エージェントはまずコードベースを読み、計画を立てます。そのうえで各ステップを確かめながら進み、実際の画面や環境で結果を確認してから止まります。mass ulw では仕事全体がエージェントのグラフになり、部分ごとに適したモデルが並列で動いて、それぞれ検証されます。Kibitzer は記憶の仕組みで、記憶を Markdown ファイルの git リポジトリとして保持します。安価な小型モデルが高価なメインエージェントの横

で過去の作業を読み、同じ失敗を繰り返しそうなときに注意を促します。ほかに、ブラウザ操作を含むスキルをタスクに応じて必要な分だけ読み込む機能や、実験的な Computer use もあります。

仕組み・アーキテクチャ

README によると、互いに依存しないツール呼び出しは一つの小さなプログラムにまとめて実行されます。そのため、20 回の読み取りも 1 往復で済みます。不正な形式のツール引数は実行前に修復されます。ビルドやデプロイの待機はポーリングではなく購読の形で扱われます。長い作業では何を確認したかの記録が残るため、セッションを再起動しても途中から再開できます。設定、スキル、プロンプトはライブでリロードされます。ファイル構成を見ると、Rust の workspace にデスクトップ操作用のクレート群（senpi-desktop-core、safety、session、engine など）があります。macOS、X11、Wayland、Win32、AT-SPI 向けのバックエンドとテスト用の fake バックエンドに分かれており、Computer use の土台と考えられます。各コンポーネント間の詳しい連携は README からは不明です。

技術構成

主要言語は TypeScript で、bun.lock や bunfig.toml があることから Bun を使って開発されています。npm パッケージ名は omo-ai です。デスクトップ操作やエンジンの信頼性確認を担う部分は Rust の Cargo workspace で書かれています。macOS 向けには objc2 系クレートと ScreenCaptureKit、Windows 向けには windows-sys と uiautomation を使っています。Linux 向けには x11rb、atspi、ashpd、reis、zbus が使われ、画面キャプチャには xcap、入力操作には enigo、クリップボードには arboard が使われています。ほかに、非同期処理の tokio、シリアライズの serde と schemars、CLI 構築の clap も含まれます。リンタには oxlint を採用しています。CI は GitHub Actions で、macOS の署名と公証、Windows や Linux のデスクトップ QA など、多数のワークフローが用意されています。

使い方・導入方法

macOS や Linux では curl -fsSL https://get.omo.dev/install.sh | bash を実行し、続けて omo を起動します。Windows の PowerShell では irm https://get.omo.dev/install.ps1 | iex を使います。インストーラは OS と CPU に合ったネイティブバイナリを ~/.local/bin に置き、チェックサムを検証します。パッケージマネージャを使う場合は bun add -g omo-ai または npm i -g omo-ai です。npm 上の omo という名前の別パッケージは無関係なので注意してください。セッション中に使える主なコマンドは次のとおりです。/thinking で思考レベル、/efforts で推論 effort を設定し、/resume で過去のセッションを開き、/login でサブスクリプションにログインします。シェルからは omo -c で前回のセッションを続けられます。設定ファイルは ~/.omo/omo.jsonc とプロジェクトの .omo/omo.jsonc で、より近いほうが優先されます。移行時は omo setup を実行し、omo doctor で状態を確認します。

注目ポイント・注意点

スター数は約 7 万で、Google、Microsoft、Vercel などの所属者に使われていると README に記載されています。ライセンス表記には注意が必要です。README のバッジは SUL-1.0、GitHub のメタデータは NOASSERTION、Rust の workspace 設定は MIT となっており、表記が揃っていません。利用前に LICENSE.md を確認するのが安全です。アプリ内の /bug で送るバグ報告には、エンジンログ、秘密情報を伏せた設定、セッションの記録、デバイス名などの識別情報が含まれると明記されています。送信前に内容を確認できる仕組みです。README には、Ultragoal と UltraQA のアイデアは oh-my-codex に由来し、概念から再

実装したとあります。また、言及しているプロジェクトやモデルとは提携関係がないと明記されています。性能に関するレビューの引用は個人の感想であり、定量的な評価は README からは不明です。

技術構成: TypeScript、Bun、Node.js / npm、Rust、tokio、serde、clap、objc2 / ScreenCaptureKit、windows-sys、uiautomation、x11rb、atspi、zbus、xcap、enigo、GitHub Actions、oxlint
向いている人: Claude Code、Codex、Cursor、OpenCode などの AI コーディングエージェントを日常的に使い、大きなタスクを最後まで任せたい開発者に向いています。複数のモデルやサブスクリプションを使い分けてコストと品質を両立したい人や、調査、移行作業、機械学習パイプラインなどを並列エージェントで一気に進めたい人にも合います。OpenCode 版の oh-my-opencode や LazyCodex からの移行ユーザーにも適しています。

テーマ 5. gsd-build/get-shit-done

https://github.com/gsd-build/get-shit-done

★ 64,182 ／ フォーク 5,432 ／ 言語 JavaScript ／ 作成 2025-12-14 ／ MIT
該当テーマ: context engineering、prompt engineering

Claude Code向けのメタプロンプティング・コンテキストエンジニアリング・仕様駆動開発システム「GSD」の旧リポジトリ（現在はアーカイブ済み）

概要

get-shit-doneは、TÂCHESによるClaude Code向けの軽量なメタプロンプティング、コンテキストエンジニアリング、仕様駆動開発（spec-driven development）システムです。スター数6万超、フォーク5千超と大きな注目を集めたプロジェクトですが、READMEによるとこのリポジトリはすでに開発の拠点ではありません。現在は「GSD Core」としてOpen GSDのリポジトリ（https://github.com/open-gsd/gsd-core）で開発が続いています。最新のソースコード、Issue、リリース、コントリビューションは移転先を利用するよう案内されています。

要点

- Claude Code向けの仕様駆動開発・メタプロンプティング基盤
- このリポジトリはアーカイブ済みで開発は停止
- 後継はopen-gsd/gsd-coreの「GSD Core」
- スター約6.4万の人気プロジェクト、ライセンスはMIT
- 主要言語はJavaScript、changesetsでリリース管理

何をするものか

リポジトリの説明によれば、GSD（get-shit-done）はClaude Codeのための軽量かつ強力なメタプロンプティング、コンテキストエンジニアリング、仕様駆動開発のシステムです。AIコーディングエージェントに渡す指示や文脈を体系的に設計し、仕様を起点に開発を進めることを狙ったものと読み取れます。ただし現在のREADMEは移転案内のみで、具体的なワークフローやコマンド体系の説明はこのリポジトリからは確認できません。詳細な機能説明はREADMEからは不明です。

リポジトリの現状（移転について）

READMEの冒頭には「GSD Has Moved」と記されており、このリポジトリはGSD開発の現役の拠点ではなくなっています。プロジェクトは「GSD Core」としてOpen GSDのリポジトリ https://github.com/open-gsd/gsd-core で継続しています。最新のソースコード、Issue報告、リリース、コントリビューションはすべて新リポジトリで行うよう明記されています。これから導入や調査をする人は、このリポジトリではなく移転先を参照するのが正しい手順です。

ファイル構成から読み取れること

ファイル一覧には.changesetディレクトリに大量の変更記録が並んでおり、changesetsによるバージョン・リリースノート管理が行われていたことが分かります。ファイル名にはphase、milestone、roadmap、workstream、worktree、statusline、installer、sdk、graphifyといった語が頻出し、フェーズやマイルストーン単位で計画を管理する仕組みや、SDK、インストーラー、git worktree連携などが存在したと推測されます。また codex、gemini、opencode、antigravity といった名前も見られ、Claude Code以外のランタイムへの対応も進められていた可能性がありますが、正確な仕様は資料からは断定できません。

技術構成と開発体制

主要言語はJavaScriptで、ライセンスはMITです。.githooks配下にpre-commitとpre-pushのフックがあり、.coderabbit.yamlからAIコードレビューツールのCodeRabbitを利用していたこと、.github/CODEOWNERSでレビュー担当を管理していたことがうかがえます。.clinerulesというファイルも含まれています。マニフェスト（package.jsonなど）は提供資料に含まれていないため、依存ライブラリやNode.jsのバージョン要件はREADMEからは不明です。

使い方・導入方法

このリポジトリのREADMEには導入手順やコマンドの記載がありません。changesetのファイル名にnpxやinstallerに関する記述が見られることから、npm経由でインストールする形式だった可能性はありますが、具体的なコマンドは資料からは確認できません。実際に利用したい場合は、移転先のopen-gsd/gsd-coreリポジトリのREADMEやリリース情報を確認してください。

注目ポイント・注意点

作成は2025年12月と新しいにもかかわらずスター6.4万超を集めており、Claude Codeを中心としたAIエージェント開発の文脈で非常に注目されたプロジェクトです。一方で、このリポジトリはアーカイブ扱いであり、ここにIssueを立てたりコードを参照して最新仕様と判断したりするのは避けるべきです。過去の変更履歴や設計の変遷を調べる資料としては有用ですが、最新情報や継続的な利用は後継のGSD Coreを前提にしてください。

技術構成: JavaScript、Claude Code、changesets、Git hooks、CodeRabbit
向いている人: Claude CodeなどのAIコーディングエージェントを使い、仕様駆動やコンテキストエンジニアリングで開発を効率化したいエンジニア向け。ただし本リポジトリはアーカイブ済みのため、新規利用者は後継のopen-gsd/gsd-coreを参照すべきで、ここは過去の変更履歴や設計の変遷を追いたい人向けの資料となる。

テーマ 6. tirth8205/code-review-graph

https://github.com/tirth8205/code-review-graph

★ 32,494 ／ フォーク 2,957 ／ 言語 Python ／ 作成 2026-02-26 ／ MIT
該当テーマ: GraphRAG

Tree-sitterでコードベースを知識グラフ化し、MCP経由でAIコーディングツールに必要最小限のレビュー文脈を渡すローカルファーストのツール

概要

AIコーディングツールが変更をレビューする際にコードベースの広い範囲を何度も読み直してしまい、トークンを浪費する問題を解決するためのツールです。リポジトリをTree-sitterで解析し、関数・クラス・importを頂点、呼び出し・継承・テストカバレッジを辺とするグラフとしてSQLiteに保存し、差分の影響範囲（blast radius）に関係するファイルだけをMCPツール経由で返します。Claude Code、Cursor、Codex、Gemini CLIなど多数のAIツールへの設定を1コマンドで行え、インクリメンタル更新やGitHub Actionによるリスクスコア付きPRレビューにも対応します。6つのOSSリポジトリでの再現可能なベンチマークを公開し、質問あたり中央値約63倍のトークン削減を示している点も特徴です。

要点

- Tree-sitterでASTを解析し、関数・呼び出し関係などをSQLiteのグラフに保存
- 変更の影響範囲（blast radius）を辿り、読むべき最小限のファイルだけを提示
- install コマンドでClaude CodeやCursorなど多数のAIツールのMCP設定を自動化
- SHA-256差分によるインクリメンタル更新で、約3,000ファイルの2ファイル編集が約2.5秒
- 30個のMCPツール、GitHub Action、VS Code拡張、マルチリポジトリdaemonを提供

何をするものか

code-review-graph は、コードベースの構造をローカルの知識グラフとして永続化し、AIコーディングアシスタントがレビューや質問応答の際に必要な部分だけを読むようにするツールです。README では、flask のコーパス全体を読むと143,594トークンかかるところ、グラフからの回答は2,196トークンで済むという例が示されています。データは .code-review-graph/ 内の単一のSQLiteファイルに保存され、外部データベースやクラウドサービスは不要です。テレメトリは無く、クラウドの埋め込みはオプトインとFAQで説明されています。CLIとMCPサーバーの両方として動作し、AIツールからはMCPツール群として、人間からは code-review-graph コマンドとして利用できます。

仕組み・アーキテクチャ

処理の流れは「リポジトリ → Tree-sitterパーサー → SQLiteグラフ → blast radius → 最小レビューセット」です。パーサーは関数、クラス、import、呼び出し箇所、継承、テストを抽出し、ノードと辺としてグラフに格納します。ファイルが変更されると、そのファイルを呼び出す側・依存する側・関連テストを辿って影響を受け得るファイル群を算出します。更新はエディタのフック、git pre-commit フック、watch モードで発火し、git diff で変更ファイルを特定、import/call 辺から依存先を探し、SHA-256ハッシュが変わったファイルだけを再パースします。後処理として Leiden によるコミュニティ検出、実行フロー検出、FTS5 インデックス作成

が行われ、ハブ・ブリッジ（媒介中心性）検出やサプライズスコアリングなどの分析にも使われます。辺にはEXTRACTED/INFERRED の2段階の信頼度が付きます。

主な機能と対応言語

Python、JavaScript/TypeScript/TSX、Go、Rust、Java、C/C++、C#、Ruby、Kotlin、Swift、PHP、Scala、Solidity、Dart、SQL、Terraform、Ansible YAML、Vue/Svelte、Jupyter/Databricks ノートブックなど幅広い言語に対応します。PHP では Composer PSR-4、Blade、Laravel の Route や Eloquent の辺、Java では Spring の DI 呼び出し解決、エンドポイント、イベント、Temporal ワークフローの辺を生成します。未対応言語は .code-review-graph/languages.toml で tree_sitter_language_pack の文法とノード種別を指定して追加できます。そのほか、sentence-transformers や Gemini、Voyage、OpenAI互換エンドポイントによる任意のセマンティック検索、D3.js による対話的可視化、GraphML・Neo4j Cypher・Obsidian などへのエクスポート、デッドコード検出、リネームプレビュー、Wiki 生成、5種類のMCPプロンプトを備えます。

技術構成

本体は Python 3.10 以上で書かれ、ビルドには hatchling を使用しています。必須依存は mcp、fastmcp、anyio、tree-sitter、tree-sitter-language-pack、pyyaml、networkx、watchdog などです。オプション依存として、ローカル埋め込み用の sentence-transformers と numpy、Google Gemini 用の google-genai、コミュニティ検出用の igraph、Python の呼び出し解決強化用の jedi、ベンチマークと SVG 出力用の matplotlib などがあります。可視化には同梱の D3.js v7 を使います。別ディレクトリ code-review-graph-vscode には TypeScript と esbuild で作られた VS Code 拡張があり、blast radius 表示やグラフ Webview などのファイルが含まれていますが、詳細は README からは不明です。開発ツールは pytest、mypy、ruff です。

使い方・導入方法

pip install code-review-graph（または pipx）でインストールし、code-review-graph install を実行すると、インストール済みのAIツールを検出してMCPサーバー設定、フック、スキル、ルールファイルへの指示を書き込みます。特定のツールだけを設定する場合は --platform cursor のように指定します。続いて code-review-graph build でコードベースを解析します。約3,000ファイルのリポジトリでコールドビルドは約40秒とされています。日常的には detect-changes --brief で変更のリスクとトークン削減量を表示し、update で差分更新を行います。フック非対応のエディタ向けには crg-daemon で複数リポジトリを監視できます。CIでは tirth8205/code-review-graph の GitHub Action を使い、PRにリスク評価付きのコメントを投稿できます。fail-on-risk を指定するとマージゲートとしても機能します。撤去は uninstall --dry-run で事前確認できます。

注目ポイント・注意点

ベンチマークは上流のSHAを固定し、Leiden のシード固定と CPU 上での決定的な埋め込みによって再現可能にしている点が丁寧です。一方で、README 自身が限界を明記しています。影響分析の recall 1.0 は、正解データが同じグラフから導出されているため循環的な上限値にすぎないこと。全コーパスとの比較はどの実エージェントも払わない上限ベースラインであること。小さな単一ファイル変更ではグラフの文脈が素のファイル読み込みより大きくなり得ること。検索順位付けや、JavaScript・Go のフロー検出には改善の余地があること。さらにFAQでは、小規模リポジトリや一回限りの質問には向かないとしています。クラウド埋め込みを使う場合は、識別子やシグネチャなどソース由来のテキストが外部に送信される点にも注意が必要です。

技術構成: Python、Tree-sitter、tree-sitter-language-pack、SQLite (FTS5)、MCP、fastmcp、anyio、networkx、watchdog、igraph (Leiden)、sentence-transformers、jedi、D3.js、TypeScript (VS Code拡張)、GitHub Actions
向いている人: Claude Code、Cursor、Codex、Gemini CLI などのAIコーディングツールを中〜大規模リポジトリで使い、トークン消費を抑えつつ変更の影響範囲を正確に把握したいエンジニアに向いています。CIでリスク評価付きのPRレビューを自動化したいチームや、依存関係・アーキテクチャ・未テスト箇所を可視化してオンボーディングやリファクタリングに役立てたい人にも有用です。小規模リポジトリや単発の質問だけの用途には向かないと README/FAQ で明記されています。

テーマ 7. muratcankoylan/Agent-Skills-for-Context-Engineering

https://github.com/muratcankoylan/Agent-Skills-for-Context-Engineering

★ 18,248 ／ フォーク 1,505 ／ 言語 Python ／ 作成 2025-12-21 ／ MIT
該当テーマ: context engineering

AIエージェント開発に必要なコンテキストエンジニアリングの知見を、Agent Skills形式でまとめたオープンなスキル集

概要

LLMのコンテキストウィンドウに入れる情報（システムプロンプト、ツール定義、取得文書、履歴、ツール出力）をどう選び、どう管理するかという「コンテキストエンジニアリング」の原則を、Agent Skills仕様に沿ったスキルとして体系化したリポジトリです。対象は、本番運用レベルのエージェントシステムを構築・最適化・デバッグする開発者です。Claude Codeのプラグインマーケットプレイスとして導入でき、Cursor、Codex、Open Pluginsにも対応しています。スキル集にとどまらず、外部研究を取り込んでスキルを更新するResearcher Operating Systemや、スキルが正しく選ばれるかを測るルーターベンチマークまで公開している点が特徴です。

要点

- 基礎・アーキテクチャ・運用など5分類、計18種類のスキルを収録
- Claude CodeのPlugin Marketplaceとしてコマンド2つで導入できる
- Cursor、Codex、Copilot CLIなどOpen Plugins対応ホストでも利用可能
- 実例システム6件と、スキルルーターのベンチマーク結果を公開
- 学術論文で静的スキル設計の先行例として引用されている

何をするものか

コンテキストエンジニアリングとは、プロンプトの書き方だけでなく、モデルの限られた注意の範囲に入るすべての情報を全体として整理する分野だと README は説明しています。コンテキストが長くなるほど、lost-in-the-middle現象やU字型の注意曲線などで性能が落ちることが知られています。そのため、目的の結果につながる情報量の多いトークンを、できるだけ少なく選ぶことが重要になります。本リポジトリは、この考え方とハーネスエンジニアリング（エージェントの動作ループの設計）の原則を、エージェントが読み込めるスキルとして提供します。特定のベンダーに依存せず、他の環境でも使える原則を重視している点が設計方針として明記されています。

主な機能（収録スキル）

Foundationalには、context-fundamentals、context-degradation、context-compressionがあります。Architecturalには、multi-agent-patterns、long-horizon-prompting、memory-systems、tool-design、filesystem-context、hosted-agentsが含まれます。Operationalには、context-optimization、self-managed-context、latent-briefing、evaluation、advanced-evaluation、harness-engineering、self-improvement-loopsがあります。このほか、開発手法としてproject-development、認知アーキテクチャとしてbdi-mental-statesを収録しています。READMEには、スキルごとに「どんな場面で有効になるか」を示した表もありま

す。たとえば、長いセッションで性能が落ちたときの原因調査にはcontext-degradation、LLM-as-a-Judgeによる評価にはadvanced-evaluationが対応します。

仕組み・アーキテクチャ

各スキルはAgent Skills仕様に従い、必須のSKILL.mdと、任意のscripts/、references/で構成されます。起動時に読み込まれるのはスキル名と説明だけで、関連するタスクのときに初めて本文が読み込まれる「Progressive Disclosure」を採用しています。これによりコンテキストの消費を抑えます。マニフェストは.plugin/plugin.jsonと.claude-plugin/marketplace.jsonにあります。researcher/ディレクトリは、外部研究をスキル変更に反映するためのファイルベースの仕組みです。ソース登録簿、ルーブリック、メカニズム登録簿、根拠の出所を記録するclaims、実行状態を管理するステートマシン、敵対的ベンチマークなどを備えています。

技術構成

主要言語はPythonです。スキル内のスクリプトは、特定の依存関係をインストールしなくても読めるPython疑似コードで書かれていると説明されています。researcherの検証・ループ処理はPythonスクリプト群（validate_repo.py、skill_health.py、loop_step.pyなど）で、macOSではlaunchdでデーモンとして動かせます。ルーターベンチマークのランナーはTypeScriptで書かれ、Cursor SDKを使います。サンプルのllm-as-judge-skillsはTypeScriptとVitestによる実装、interleaved-thinkingはPythonパッケージです。CIはGitHub Actions（validate.yml）で動きます。

使い方・導入方法

Claude Codeの場合は、まず /plugin marketplace add muratcankoylan/Agent-Skills-for-Context-Engineering でマーケットプレイスを登録します。次に /plugin install context-engineering@context-engineering-marketplace を実行すると全スキルが入り、タスクの内容に応じて自動で有効になります。Cursorでは、Cursor Plugin Directoryからインストールする方法が推奨されています。個別のスキルだけ使う場合は、スキルのフォルダを .claude/skills/、.cursor/skills/、.codex/skills/、.agents/skills/ などにディレクトリごとコピーします。SKILL.mdを1枚のファイルにまとめると references/ への相対パスが壊れるため、避けるよう注意書きがあります。

注目ポイント・注意点

スキルルーターについて、4つの最先端モデルで各600回の呼び出しによるベンチマークを行い、結果を公開しています。説明文を書き直した結果、context-fundamentalsのtop-1精度は0.255から0.489に上がりました。全体を強化した後のtop-1精度は、gemini-3.1-proで0.920、claude-opus-4-7で0.840です。researcherのループは有料LLMを呼ばず、外部への書き込みもしません。mergeとpushは人間が行う設計です。一方で、スキルの多くは概念と原則が中心のため、実装は利用者が自分のフレームワークに合わせて行う必要があります。各スキルの詳しい内容はファイル一覧だけでは確認できません。

技術構成: Python、TypeScript、Markdown（SKILL.md / Agent Skills仕様）、Cursor SDK、Vitest、GitHub Actions、launchd、JSONL / YAML / JSON Schema
向いている人: Claude Code、Cursor、Codexなどでエージェントを開発している人、マルチエージェントや長時間

動く自律エージェントの設計・評価・デバッグに取り組むエンジニアに向いています。コンテキスト管理、メモリ設計、LLM-as-a-Judgeによる評価などを体系的に学びたい学生や研究者にも適しています。

テーマ 8. Orchestra-Research/AI-Research-SKILLs

https://github.com/Orchestra-Research/AI-Research-SKILLs

★ 14,112 ／ フォーク 987 ／ 言語 TeX ／ 作成 2025-11-03 ／ MIT
該当テーマ: GRPO

Claude CodeやCodex、Gemini CLIなどのAIエージェントに、AI研究の全工程を自律的に進めるための知識を与えるSkillsライブラリ

概要

AI研究者が仮説の検証よりもインフラのデバッグに時間を取られているという課題に対し、AIエージェントが文献調査から実験、論文執筆までを自律的に進められるようにするためのSkills集です。Megatron-Core、vLLM、TRLなど主要フレームワークごとに専門知識をまとめたスキルを、23カテゴリ・98個用意しています。中心となるautoresearchスキルは二重ループ構成で研究全体を統括し、必要に応じて各ドメインスキルに処理を振り分けます。npxによるワンコマンドのインストーラで、複数のコーディングエージェントにまとめて導入できる点も特徴です。

要点

- 23カテゴリ・98スキルでAI研究の全工程をカバー
- autoresearchスキルが研究全体を統括し各スキルへ振り分ける
- 各スキルはSKILL.mdとreferencesで構成される段階的開示型
- npxコマンド一つで複数のエージェントへ導入できる
- MITライセンス。ただし参照先ライブラリのライセンスは個別に確認が必要

何をするものか

AIエージェントが自律的にAI研究を進めるためのスキルライブラリです。README では、文献調査、アイデア出し、実験の実行、論文執筆までを一通り扱えることを目標に掲げています。全体は二つの層に分かれます。一つは研究を統括する層で、autoresearch、ideation、paper writing が含まれます。もう一つはエンジニアリング層で、学習、評価、デプロイといった各段階の技術を扱います。Claude Code、Codex、Gemini CLI などのコーディングエージェントにこのスキル群を読み込ませることで、エージェントを研究エージェントとして使えるようにする、という位置づけです。

主な機能とカテゴリ構成

スキルは23カテゴリに分類されています。Model Architecture（LitGPT、Mamba、NanoGPT、RWKV、TorchTitan）、Fine-Tuning（Axolotl、LLaMA-Factory、PEFT、Unsloth）、Post-Training（TRL、GRPO、OpenRLHF、SimPO、verl、slime、miles、torchforge）、Distributed Training（DeepSpeed、FSDP、Accelerate、Megatron-Core など）、Inference（vLLM、TensorRT-LLM、llama.cpp、SGLang）があります。さらに Mech Interp、RAG、Multimodal、MLOps、Safety なども揃っています。論文執筆側には ML Paper Writing があり、NeurIPS、ICML、ICLR などのLaTeXテンプレートと引用検証を備えます。新しいカテゴリの Agent-Native Research Artifact は、研究成果を主張・証拠・探索グラフに構造化するためのものです。

仕組み・アーキテクチャ

中核となる autoresearch スキルは二重ループ構成です。内側が最適化ループ、外側が統合ループで、研究ライフサイクル全体を管理しながら適切なドメインスキルへ処理を振り分けます。継続的に動かす手段として、Claude Code の /loop と OpenClaw の heartbeat（cron）に対応しています。作業領域には research-state.yaml、findings.md、research-log.md、experiments/ などが置かれ、findings.md はセッションをまたいで使う記憶として機能します。各スキルの構成は共通で、50〜150行程度の SKILL.md がクイックリファレンスを担います。詳しい資料は references/ に置かれ、API、チュートリアル、実際のGitHub issue、リリース履歴などが含まれる段階的開示型の設計です。

使い方・導入方法

人間が導入する場合は、npx @orchestra-research/ai-research-skills を実行すると対話型インストーラが起動します。インストーラは導入済みのエージェントを自動検出し、スキルを ~/.orchestra/skills/ に配置したうえで各エージェントへシンボリックリンクを張ります。Windows ではシンボリックリンクの代わりにコピーを使います。サブコマンドは list と update が用意されています。AIエージェントに任せる場合は、公式サイトの welcome.md を読んで指示に従うよう伝えるだけで導入できます。Claude Code では /plugin marketplace add でマーケットプレイスを追加し、fine-tuning@ai-research-skills のようにカテゴリ単位で入れることも可能です。

技術構成と運用

リポジトリの中身は主にMarkdownのドキュメントです。主要言語はTeXと表示されており、論文テンプレートが含まれていることが理由と考えられます。インストーラは packages/ai-research-skills にあるnpmパッケージで、依存ライブラリには chalk、inquirer、ora が挙げられています。GitHub Actions のワークフローは三つあります。check-inventory.yml はドキュメントに書かれたスキル数と実際の SKILL.md の数が一致するかを検証します。sync-skills.yml は Orchestra のマーケットプレイスへの同期、publish-npm.yml はnpmへの公開を担当します。ドキュメントの作成には、ドキュメント自動収集ツールの Skill Seeker と Claude Code が使われています。

注目ポイント・注意点

注目点として、autoresearch スキルを使ってエージェントがほぼ全工程を書き上げたとされる論文デモが2本公開されています。一つは Norm Heterogeneity で、もう一つは RL Brain Scan です。注意点もあります。README 内のスキル数の表記が箇所によって87、98などと揺れています。また、ファイル一覧には一覧表に載っていない swanlab、a-evolve、ml-training-recipes も含まれています。ライセンスはMITですが、各スキルが参照するライブラリには別のライセンスが適用される場合があるため、個別に確認が必要です。スキルの品質をどう評価したかや、エージェントごとに動作がどう異なるかは README からは不明です。

技術構成: Markdown (SKILL.md)、TeX / LaTeX、Node.js / npm (npx インストーラ)、chalk、inquirer、ora、GitHub Actions、Python（一部テンプレート・サンプル）、Claude Code Plugin Marketplace
向いている人: Claude Code、Codex、Gemini CLI、Cursor などのコーディングエージェントでAI研究や機械学習の開発を効率化したい研究者、MLエンジニア、学生に向いています。具体的な用途は、ファインチューニング、RLに

よるポストトレーニング、分散学習、推論の最適化、論文執筆などです。エージェントに主要フレームワークの正確な使い方を参照させたいチームや、自律的な研究エージェントを試してみたい人にも役立ちます。

テーマ 9. adongwanai/AgentGuide

https://github.com/adongwanai/AgentGuide

★ 11,450 ／ フォーク 1,073 ／ 言語 MDX ／ 作成 2025-11-03
該当テーマ: RAG、GraphRAG、GRPO

JavaGuideを手本にした、AI Agent開発の学習から大模型関連職の就職面接対策までを網羅する中国語のオープンソース知識ベース

概要

AgentGuideは、AI Agentの工学・研究・就職活動を一体的に扱う中国語のオープンソース学習ガイドで、「Agent = Harness + Model」を掲げています。ネット上に散らばる良質な講座・論文・OSSを引用して体系的な学習ルートにつなげ、各トピックに「面接でどう問われるか」「履歴書にどう書くか」を添える就職志向が特徴です。開発職（8〜12週）と算法職（10〜15週）の2ルートに加え、1500問以上とされる面接問題集、Agent Harness・Context Engineering・Eval・Agentic RLなど2026年時点の前沿トピックを収録しています。単なるリンク集ではなく、段階ごとの成果物と実行可能なTodo Listで「何を作ったか」を示せる状態を目指す点が新しいところです。

要点

- JavaGuide的な位置づけのAI Agent学習・就活ガイド
- 開発職と算法職で分かれた学習ルートと期間目安
- 1500問超の面接問題集と大手企業の面接体験談
- Harness・Context Engineering・MCP・Evalを重視
- 実戦プロジェクトは多くが「近日公開」で未完成

何をするものか

AgentGuideは、AI Agent分野への転職や新卒就職を目指す人向けに、学習ロードマップ・技術解説・実戦プロジェクト・面接対策をまとめた中国語の知識ベースです。READMEでは「做得出、跑得稳、测得准、讲得清（作れる、安定して動く、正しく測れる、説明できる）」を軸に内容を構成すると述べています。特定フレームワークに縛られず、Agent LoopやHarness、Context EngineeringとMemory、ToolsとMCP・Skills・A2A、RAG、Eval・Observability・Safety、SFTやDPO/GRPOなどのPost-trainingまでを一通り扱います。既存の良質な資料は引用し、車輪の再発明はしない方針です。

主な機能・コンテンツ

中心となるのは6ステップの学習パスです。志望職種の決定、内定獲得の方法論、職種別ルート、実戦プロジェクト、体系的な技術学習、面接対策の順に進みます。技術学習はL1基礎認知層、L2開発実装層、L3高度最適化層の12モジュールで構成されています。面接対策では理論題98問、RAG題22問、Agent題52問、手書きコーディング34問、アルゴリズム題503問などを分類して収録しています。さらに職種別の専門題、大手企業16社分の面接事例、小紅書の面接体験談の整理、給与交渉やHR面接の攻略もあります。加えて8段階の学習成果物表、プロジェクト実装の5ステップ、履歴書の三次元表現法など、すぐ実行できる形の指針が並んでいます。

リポジトリ構成

ファイル一覧を見ると、本体はMarkdown文書です。docs配下に00-getting-startedから06-research-frontiersまでの章があり、resources配下に論文PDF、Agent Memory関連資料、業界レポートが置かれています。projectsにはpaper-agent、travel-agent、web-agentのREADME、examplesには最小Agent Loopの解説、eval-cases.jsonl、trace-schema.json、ツールカードのテンプレートがあります。appsには2つのAstro製サブアプリがあります。InterviewGuideは面接問題を検索・フィルタできるサイトで、TypeScriptのデータビルドスクリプトとVitestのテストを持ちます。ai-research-ebookはMDXで書かれた研究支援の電子書籍です。GitHub Actionsでは、CI、GitHub Pagesへのデプロイ、サイトのヘルスチェックが定義されています。

技術構成

主要言語はMDXと表示されていますが、大部分はMarkdownのドキュメントです。サブアプリのInterviewGuideとai-research-ebookは、Astro、Tailwind CSS、TypeScriptで作られており、InterviewGuideはテストにVitestを使っています。docs/04-interview/examplesには、履歴書の表現をチェックするPythonスクリプトもあります。解説の対象になっている技術は、LangGraph、OpenAI Agents SDK、AutoGen、CrewAI、AgentScope、Dify、n8n、MCP、Milvus、Qdrant、Chroma、RAGAS、DeepEval、LLaMA-Factoryなど多岐にわたります。ただし、これらはリポジトリが依存しているライブラリではなく、学習対象として紹介されているものです。

使い方・導入方法

基本的には、GitHub上かGitHub Pagesのサイト（adongwanai.github.io/AgentGuide）でドキュメントを読む使い方になります。READMEのクイックスタートでは、まずdocs/01-theory/01-what-is-agent.mdを読み、算法系の人は続けてdocs/01-theory/04-react-framework.mdに進むよう案内しています。「How To Use」の表では、自分の状態ごとに入口が示されています。零基礎なら6ステップパス、プロジェクトを作りたいなら実戦プロジェクト、面接準備中なら問題集から始めます。サブアプリのビルド手順はREADMEからは不明です。貢献する場合は、Fork、ブランチ作成、PRという一般的な流れが案内されています。

注目ポイント・注意点

スターは1万を超え、Agent Harness、Coding Agent、Agentic RLといった最新テーマにも追従している点は注目に値します。一方で、READMEに載っている論文分析Agentや旅行計画Multi-Agentなど多くの実戦プロジェクトは、学習パスの各項目が「即将推出（近日公開）」のままです。「完全なコード実装」をうたう記述と実態には差がある可能性があります。年収レンジや「8〜10週で内定」といった数値は作者の主張であり、中国の就職市場を前提にしている点にも注意が必要です。ライセンスは不明と表示されているため、転載や二次利用の前に確認してください。コミュニティへの参加は公众号経由が中心です。

技術構成: Markdown、MDX、Astro、TypeScript、Tailwind CSS、Vitest、Python、GitHub Actions、GitHub Pages
向いている人: AI Agent開発職、算法職、RAGエンジニアを目指す学生や転職者に向いています。LLMアプリ開発の知識を体系化したいエンジニアや、大模型関連職の面接対策をしたい人にも役立ちます。中国語が読めて、中国の就職市場を念頭に置いた情報でも参考にできる人に最適です。

テーマ 10. MakazhanAlpamys/Soup

https://github.com/MakazhanAlpamys/Soup

★ 9,808 ／ フォーク 1,616 ／ 言語 Python ／ 作成 2026-02-20 ／ Apache-2.0
該当テーマ: LLM fine-tuning、LoRA LLM

YAML一つでLLMのファインチューニングから書き出しまでを行うCLIで、4GB GPUで8Bモデルを学習できるレイヤーストリーミングを備える

概要

Soupは、LLMのファインチューニングやポストトレーニングにつきまとうインフラ設定の手間を、YAML設定ファイル一つとコマンド一つにまとめるPython製CLIツールです。SFT、DPO、GRPO、KTO、ORPOなど多様な学習手法、LoRA系のPEFT、量子化、GGUF書き出し、OpenAI互換サーバーまでを一つのツールで扱えます。目玉はベータ機能のレイヤーストリーミングで、凍結したベースモデルをホストRAMに置いてデコーダ層を一つずつGPUへ送ることで、RTX 3050 Laptop 4GBでもLlama-3.1-8BのLoRA学習ができると、実測値付きで示されています。性能値は benchmarks/ に計測記録として公開され、論文の誤った説明も新しい版で撤回するなど、検証の透明性を重視している点も特徴です。

要点

- soup init と soup train の2コマンドで学習を始められる
- レイヤーストリーミングで4GB GPUでも8BモデルのLoRA学習が可能（BETA）
- SFT/DPO/GRPO/PPO/KTOなど多数の学習タスクに対応
- GGUF書き出し、OpenAI互換サーバー、Web UIまで一つのツールで扱える
- 実測ベンチマークと論文で主張を検証・公開している

何をするものか

SoupはLLMのファインチューニングとポストトレーニングを、YAMLの設定ファイル一つで実行するコマンドラインアプリケーションです。READMEは、チームが時間の30〜50%をインフラとの格闘に使っていると指摘し、その負担を減らすことを目的に掲げています。SSHでGPUサーバーに入る必要がないこと、バッチサイズ・GPU検出・量子化の自動設定、QLoRAによるローカルGPUでの学習が特徴です。AutoModelForCausalLMで読み込めるHugging Face Hubのテキスト生成モデルであれば、設定を変えずに使えるとされています。Llama 3.x/4、Qwen 2.5/3、Gemma 3、Mistral、DeepSeek、Phi-4など100以上のモデルについては、すぐ使えるレシピが用意されています。

主な機能

学習タスクはSFTのほか、DPO、GRPO、PPO、KTO、ORPO、SimPO、IPO、BCO、事前学習、蒸留、ビジョンや音声などに対応します。PEFTではDoRA、LoRA+、rsLoRA、VeRA、PiSSA、GaLoreなどを扱えます。学習後は soup chat で対話テスト、soup merge でLoRAをベースモデルに統合、soup export でGGUF（Ollama / llama.cpp向け）、ONNX、TensorRTなどに書き出せます。soup serve でOpenAI互換のAPIサーバーを立てられ、soup ui ではブラウザ上のダッシュボードを使えます。ほかにも、データの重複除去やPII除去を行う soup

data、評価用の soup eval、環境診断の soup doctor、設定なしで学習する soup autopilot、監査ログやBOMなどのガバナンス機能が揃っています。

仕組み：レイヤーストリーミング

stream_layers: true で有効にするオプトインのベータ機能です。凍結されたベースモデルをVRAMに常駐させず、ホストRAMに固定（pin）しておき、デコーダ層を一つずつGPUへ転送しながら学習します。v0.72.2でのRTX 3050 Laptop 4GBの計測では、Llama-3.1-8B-Instruct + NF4、LoRA、batch 1、seq 512でピーク3.32GB、119.6 tok/sでした。通常の常駐実行とビット単位で一致したと報告されています。修正後の別環境の計測は2.40GB、208.6 tok/sですが、READMEはこの差の大部分をクロック差によるもので、高速化ではないと明記しています。論文v3では「ボトルネックはホスト→デバイス転送」という以前の説明を撤回しており、最大の固有コストは層ごとのNF4逆量子化（9.8%）だとしています。無料のColab T4で4GB制限をかけて確かめられるノートブックも用意されています。

技術構成

本体はPythonで書かれ、対応バージョンは3.11〜3.13です。学習スタックには torch、transformers、peft、trl、datasets を使い、これらは [train] extra でインストールします。素の soup-cli は、PyTorchを含まない軽量なCLI・設定・データツールだけの構成です。高速化オプションとして unsloth バックエンド（[fast]）があり、Apple Silicon向けには MLX バックエンドがあります。マルチGPUでは DeepSpeed や FSDP に対応します。設定項目は config/schema.py に一元的に定義されており、v0.75以降は未知のキーを書くと設定の読み込みが失敗します。DockerイメージはCUDA 12.8 + Ubuntu 24.04をベースに、GHCRで配布されています。テストには pytest、リンタには ruff を使い、CIにはGitHub Actionsを使っています。

使い方・導入方法

推奨されるインストール方法は pipx install "soup-cli[train]" または uv tool install soup-cli です。仮想環境内では pip も使えます。extras指定はダブルクォートで囲むよう注意書きがあります。手順は、soup init --template chat で設定を作り、soup train --config soup.yaml で学習し、soup chat --model ./output で試すという流れです。テンプレートには chat、code、tool-calling、reasoning、vision、kto、orpo、moe、longcontext などがあります。データ形式はAlpaca、ShareGPT、ChatML、選好ペアなどで、JSONL、CSV、Parquetなどから自動判別されます。Dockerを使う場合は docker run --gpus all でマウントしたディレクトリに対して train を実行し、非対話実行では --yes が必要です。

注目ポイント・注意点

性能値はすべて benchmarks/ に計測記録として公開されており、失敗した計測や破棄した数値も含まれています。READMEは数値の比較条件にも丁寧に注釈を付けています。一方で、レイヤーストリーミングはまだBETAです。v0.77.0では以前は警告にとどまっていた設定を拒否するようになったため、0.76.xから上げる場合は先に soup doctor --config で確認するよう案内されています。VRAMの事前見積もりは3.5B程度までの密なモデルで較正されたもので、その範囲外では警告として扱われます。CPUでの学習は実験的で非常に遅いとされています。マルチGPUや大規模モデルの検証は、作者の4GBノートPCでは実施できないため、コミュニティのハードウェア提供を募っています。テレメトリは既定で無効です。

技術構成: Python、PyTorch、transformers、peft、trl、datasets、bitsandbytes (NF4/QLoRA)、Unsloth、MLX、DeepSpeed / FSDP、llama.cpp / GGUF、Docker (CUDA 12.8)、pytest、ruff、GitHub Actions
向いている人: 手元のコンシューマGPU（4〜8GBクラスを含む）やApple SiliconでLLMをファインチューニングしたい個人開発者や学生に向いています。設定地獄を避けてSFTやDPOなどを素早く回したい機械学習エンジニアや、学習したモデルをGGUF化してOllamaで使いたい人にも適しています。監査・コンプライアンス機能を必要とするチームにも向いています。

テーマ 11. Gen-Verse/OpenClaw-RL

https://github.com/Gen-Verse/OpenClaw-RL

★ 5,742 ／ フォーク 614 ／ 言語 Python ／ 作成 2026-02-26 ／ Apache-2.0
該当テーマ: RLHF、GRPO

日常の会話フィードバックを学習信号に変え、OpenClaw上のエージェントを完全非同期の強化学習で継続的に最適化するフレームワーク

概要

OpenClaw-RLは、自前でホストしたLLMをOpenAI互換APIとしてOpenClawに接続し、実際のマルチターン会話を横取りして、利用を止めずにバックグラウンドで方策を更新し続ける強化学習フレームワークです。事前収集データでバッチ学習する従来のLLM向けRLとは異なり、次のユーザー発話やツール・環境の応答を「次状態」の信号として自動的に報酬化するため、人手によるラベル付けが不要です。学習手法はBinary RL（GRPO）、On-Policy Distillation（OPD）、両者を組み合わせたHybrid方式の3つを備えています。個人向けエージェントの最適化に加えて、terminal・GUI・SWE・tool-callといった実環境エージェントの大規模RLにも同じ基盤で対応しています。

要点

- サービング・ロールアウト収集・PRM評価・学習を独立させた完全非同期の4コンポーネント構成
- 会話中の次状態フィードバックを自動で報酬化するため、人手ラベル付けが不要
- Binary RL（GRPO）、OPD、Hybridの3つの学習手法を搭載
- slimeベースのローカルGPU学習と、TinkerやFireworks AIによるクラウド学習の両方に対応
- terminal・GUI・SWE・tool-callの実環境エージェントRL（Track 2）も提供

何をするものか

OpenClaw-RLは、OpenClawで使うパーソナルAIエージェントを「話しかけるだけで」学習させるための強化学習フレームワークです。自前のモデルをOpenAI互換APIとして公開し、OpenClawからの実際の会話をそのまま学習データとして取り込みます。ユーザーが普段どおりエージェントを使っている間も、裏側では軌跡の収集、報酬計算、モデル更新が継続して行われます。READMEでは「使えば使うほどエージェントが良くなる」と説明されています。もう一つの柱であるTrack 2では、同じ非同期RL基盤を使い、terminal、GUI、SWE、tool-callといった実世界の汎用エージェントを大規模並列環境で訓練できます。技術レポートはarXiv（2603.10165）で公開されており、HuggingFace Daily Papersで1位になったと記載されています。

主な機能と3つの学習手法

Binary RL（GRPO）では、Process Reward Model（PRM）が次状態のフィードバックを基に各ターンを採点します。得られたスカラー報酬をGRPOのアドバンテージ推定とPPO風のクリップ付き代理損失で使います。OPD（On-Policy Distillation）では、次状態から有用な事後情報が読み取れる場合に、judgeモデルがテキストのヒントを抽出します。そのヒントを元のプロンプトに加えて「強化された教師」を作り、生徒とのトークン単位の対数確率差を方向性のあるアドバンテージとして使います。Hybrid方式は両者を統合したもので、READMEでは単独の手法より強く安定した最適化ができるとされ、推奨手順としてopenclaw-combineが案内

されています。このほか、LoRA学習、複数人のフィードバックによる単一モデルの最適化、Qwen3.5（4B/9B/27B、テキストとマルチモーダル）にも対応しています。

仕組み・アーキテクチャ

中核は、エージェントのサービング、ロールアウト収集、PRMやjudgeによる評価、方策学習の4つを独立した非同期ループに分けた構成です。互いにブロックしないため、学習中もモデルはリクエストに応答し続けます。評価も新しい対話と並行して進みます。システムはマルチターンの対話をセッション単位の軌跡に整理し、APIメッセージを学習対象のmain-lineターンと対象外のsideターンに分類します。次のユーザー発話、環境出力、ツール応答を自然な「次状態」の信号として扱い、必要に応じて多数決で採点を頑健にしたうえで、準備ができたサンプルから順にトレーナーへ送ります。Track 2の各環境では、terminalはstdoutや終了コード、GUIは画面状態の差分、SWEはテスト結果やdiff、tool-callは戻り値やエラートレースを次状態の信号として使います。ポリシーモデル、PRM、トレーナーはすべて自前のインフラで動かせるため、会話データを外部APIに出さずに済む設計です。

技術構成

ベースのRLフレームワークはTHUDMのslimeで、リポジトリにはMegatron-LMも同梱されています。ロールアウトや推論にはSGLangを使っており、トピックやAPIキーの設定項目（SGLANG_API_KEY）からも確認できます。動作要件はCUDA 12.9とPython 3.12で、デフォルト構成はGPU 8枚です。GPUの割り当てはNUM_GPUS、ACTOR_GPUS、ROLLOUT_GPUS、PRM_GPUSで調整できます。ジョブはray job submit形式で起動します。リポジトリは手法ごとに独立したフォルダで構成されています。主なものはopenclaw-combine、openclaw-opd、openclaw-test、terminal-rl、gui-rl、swe-rl、toolcall-rl、OpenClaw用のextensions/rl-training-headersです。クラウド側ではTinkerとFireworks AIの利用が案内されています。マニフェストファイルは提示されていないため、依存パッケージの詳細はREADMEからは不明です。

使い方・導入方法

個人エージェント向けの最短手順は、slimeディレクトリに移動して bash ../openclaw-combine/run_qwen3_4b_openclaw_topk_select.sh を実行することです。起動すると、http://<HOST_IP>:30000/v1でOpenAI互換APIが立ち上がります。ポートは環境変数PORTで変更できます。次に、手元のOpenClawに拡張機能rl-training-headersを入れ、openclaw.jsonの models → providers にこのエンドポイントを登録します。設定項目はbaseUrl、apiKey、モデルID、contextWindowなどです。あとはチャットするだけで学習が進みます。Tinkerを使う場合は、localhostを指す別の設定例も示されています。Track 2では、terminal_qwen3_8b_rl.sh、gui_qwen3vl_8b_rl.sh、run_swe_rl_32b_remote_8nodes.sh、retool_qwen3_4b_rl.shといったスクリプトが用意されており、詳しい手順は各フォルダのREADMEで案内されています。

注目ポイント・注意点

会話そのものを継続的な学習信号に変え、オンラインで個人向けに最適化していく点が、従来のバッチ型RLHFとの大きな違いです。評価例として、宿題をAIでこなしたい学生と、具体的で丁寧なコメントを求める教師の双方がOpenClawを使うシナリオも用意されています。一方で、ローカル学習にはデフォルトでGPU 8枚が必要で、SWE向けには8ノード構成のスクリプトもあり、計算資源のハードルは高めです。GPUがない場合はTinkerやFireworksを使う選択肢がありますが、TinkerはLoRAのみ対応とされています。READMEは、会話

に機微な個人情報を含めないこと、APIキーをプロンプトやログに残さないことを注意喚起しています。コア部分（slime、Megatron-LM、openclaw）は改変せず、拡張ポイントから新手法を追加する方針も明示されています。

技術構成: Python、slime、Megatron-LM、SGLang、Ray、OpenClaw、Tinker、Fireworks AI、LoRA、GRPO、Qwen3 / Qwen3.5 / Qwen3-VL、CUDA 12.9
向いている人: LLMエージェントの強化学習（GRPO、蒸留、PRM）を研究するエンジニアや研究者、自前GPUやクラウドで個人向けエージェントを継続的に改善したいOpenClawユーザーに向いています。terminal・GUI・SWE・tool-callなど実環境のエージェントRL基盤を探している開発者や、エージェントRLを学ぶ大学院生にも参考になります。

テーマ 12. walkinglabs/hands-on-modern-rl

https://github.com/walkinglabs/hands-on-modern-rl

★ 4,823 ／ フォーク 352 ／ 言語 Jupyter Notebook ／ 作成 2026-04-10 ／ NOASSERTION
該当テーマ: RLHF、GRPO

CartPoleからRLHF・GRPO・Agentic RLまでを一貫して学べる、実験重視のオープンな強化学習教材

概要

hands-on-modern-rl は、マルコフ決定過程や価値関数といった強化学習の基礎から、PPO、RLHF、DPO、GRPO、RLVR、さらにツール利用エージェントやマルチモーダルRLまでを、全7部26章で体系的に学べるオープンコースウェアです。PythonとPyTorchの基礎がある学生・研究者・エンジニアを対象としています。各章は「問題・手法・実験・振り返り」の流れで構成されています。古典RLとLLMの事後学習を同じ逐次意思決定の枠組みでつなぎ、数式の横に実装を並べ、実際の学習曲線で主張を検証する点が特徴です。

要点

- 全7部26章で古典RLからAgentic RL・VLM RLまでを網羅
- PPO・DPO・GRPOの導出と実装コードを並べて解説
- ModelScope上でブラウザから学習を試せるNotebookを用意
- VitePressサイトとPDFで中英両言語を提供
- AI支援で作成され未レビューの部分があり、ライセンスは非商用

何をするものか

強化学習を、CartPoleでの試行錯誤から始めて、LLMのアラインメントやエージェントの訓練まで一本の学習パスとして学ぶための教材リポジトリです。前半の第1〜12章ではMDP、価値関数、ベルマン方程式、DQN、方策勾配、Actor-Critic、PPO、連続制御、オフラインRL、模倣学習などの古典・深層RLを扱います。後半の第13〜26章では、RLHF、DPO、GRPO、RLVR、推論モデル、プロセス報酬、ツール利用、コーディングエージェント、Deep Research、GUIエージェント、VLM RL、報酬ハッキングと評価、セルフプレイを扱います。全体を貫く問いは三つです。意思決定過程をどう表現するか、結果を過去の行動にどう帰属させるか、方策が本当に改善したかをどう判定するか、です。

主な機能・ハンズオン例

README では代表的な実験として8つが紹介されています。CartPoleのPPO、Atari PongのDQN、LunarLanderのDQN、BipedalWalkerのA2C、DPOによる選好チューニング、veRLのPPOで実行可能なI/Oテストを報酬にするCode RLVR、小規模なオフライン検索方策をREINFORCEで学習するDeep Research、EasyR1のGRPOによるGeoQA幾何推論です。これに加えて、14本の章別自己完結Notebookと12本のStudio連携Notebookがあります。さらに付録として、SFT、PPO、DPO、GRPO、DAPOなどのコンパクトな実装集、数学の基礎、学習システムのデバッグやGPU時間見積もりといった工学的な参考資料も収録されています。

構成・アーキテクチャ

リポジトリは大きく三つに分かれています。docs/ は VitePress で構築された講義本文で、テーマ用の Vue コンポーネントやナビゲーション設定を含みます。code/ は章に対応した実行可能な Python サンプルで、章ごとに requirements.txt が用意されています。scripts/ には保守・検証用スクリプトが置かれています。code/online-experiments には ModelScope Studio と同じ学習ランタイムを読み込む Notebook がまとめられており、パラメータの公開、学習ログの表示、チェックポイント評価の可視化、学習済み方策のリプレイ表示を行えます。GitHub Actions のワークフローとしては、デプロイ、コンテンツ品質チェック、PDFリリースの3つがあり、中国語版と英語版のPDFはCIで自動ビルドされます。

技術構成

教材コードは主に Python と PyTorch で書かれています。ファイル名や README からは、Stable-Baselines3（dqn_atari_sb3.py など）、Gymnasium、veRL、EasyR1 が使われていることが読み取れます。オンライン実験では、ViZDoom、Atari/ALE、MiniGrid、JAX MinAtar、ManiSkill、MineStudio、Unity ML-Agents、AI2-THOR などの環境が登場します。chapter04_dqn には PyBoy を使ったポケモン赤の DQN スクリプトも含まれています。LLM 実験では Qwen2.5-0.5B や Qwen Coder 向けの実行シェルスクリプトがあります。ドキュメントサイトは Node.js 18 以上と VitePress で構築され、整形には Prettier が使われています。各ライブラリの詳細なバージョンは README からは不明です。

使い方・導入方法

最も手軽なのは、公開サイト walkinglabs.github.io/hands-on-modern-rl をブラウザで読む方法です。ローカルでサイトを起動する場合は、リポジトリを clone してから npm install と npm run dev を実行し、通常は http://localhost:5173 を開きます。コードを動かす場合は、code ディレクトリで python -m venv .venv により仮想環境を作り、pip install -r requirements.txt で依存関係を入れます。インストールを小さく抑えたいときは、pip install -r chapter01_cartpole/requirements.txt のように章ごとの依存だけを入れ、python chapter01_cartpole/1-ppo_cartpole.py を実行します。LLM、VLM、重いシミュレータを使う章は、GPU、モデルのダウンロード、追加のシステムライブラリが必要になる場合があるため、まず第1章から始めることが推奨されています。

注目ポイント・注意点

README には、本コースがAIの支援を受けて作成され、まだ完全にはレビューされていないため、事実誤りや動かないコードが含まれる可能性があると明記されています。構築中と表示された章は避け、それ以外の章を中心に読むことが勧められています。ライセンスは CC BY-NC-SA 4.0 で、非商用での利用・改変は可能ですが、改変物を配布する場合は同じライセンスで公開する必要があります。オンライン実験のうち 03、08、10、11、12 は ModelScope の xGPU Notebook が必要です。ロードマップでは Agentic RL プロジェクト、Unity ベースの身体性RL、Diffusion RL などが今後追加予定とされています。コミュニティ窓口は WeChat グループで、日本語版は用意されていません。

技術構成: Python、PyTorch、Jupyter Notebook、Gymnasium、Stable-Baselines3、veRL、EasyR1、VitePress、Vue、Node.js、ModelScope、GitHub Actions
向いている人: 機械学習の基礎とPython・PyTorchの経験があり、強化学習を体系的に学びたい学生・研究者・エンジニアに向いています。特に、PPOからRLHF、DPO、GRPO、RLVRへのつながりを理解したいLLM事後学習の実務

者や、ツール利用・コード・マルチモーダルエージェントの強化学習を設計したい人に適しています。また、環境構築なしにブラウザで古典RL実験を試したい初学者にも向いています。非商用の教育用途であれば、教材として二次利用することも可能です。

テーマ 13. amitshekhariitbhu/ai-engineering-interview-questions

https://github.com/amitshekhariitbhu/ai-engineering-interview-questions

★ 3,588 ／ フォーク 638 ／ 言語 Markdown ／ 作成 2026-03-21 ／ Apache-2.0

該当テーマ: LLM fine-tuning

AIエンジニア職の面接対策として、LLM・RAG・エージェント等の頻出質問と解説記事へのリンクをまとめたチートシート集

概要

AI Engineer、LLM Engineer、MLOps/LLMOps Engineer などの面接に向けて、出題されやすい質問をカテゴリ別に整理した Markdown のみのリポジトリです。各質問には主に Outcome School のブログ記事や YouTube 動画、LinkedIn 投稿への回答リンクが付けられており、回答本文はリポジトリ内ではなくリンク先にあります。LLM の基礎から RAG、AI エージェント、MCP、ファインチューニング、推論基盤、評価、安全性、システム設計まで幅広く網羅しています。「〜が起きたらどう直すか」というシナリオ型の質問が多く、知識の確認だけでなく実務的な判断力を問う形式になっている点が特徴です。

要点

- AIエンジニア系職種向けの面接質問を15カテゴリで整理
- 回答の多くは Outcome School のブログや動画への外部リンク
- 障害・不具合を想定したシナリオ型質問が豊富
- DeepSeek-V4、MCP、Agent Skills など新しい話題も収録
- 中身は README.md のみで、Apache-2.0 ライセンス

何をするものか

AI エンジニアリング職の面接準備用チートシートです。対象として AI Engineer、Gen AI Engineer、LLM Engineer、Agentic AI Engineer、Forward Deployed Engineer、AI Solutions Architect、AI Platform Engineer、MLOps Engineer、LLMOps Engineer などが挙げられています。作成・管理しているのは Outcome School の創設者 Amit Shekhar 氏です。README には「新しい質問と回答を随時追加していく」と書かれており、継続的に更新される前提のリストです。ファイル構成は README.md、LICENSE、バナー画像だけで、コードは含まれません。

収録カテゴリと内容

目次には Must Know、LLM Fundamentals、Prompt Engineering、RAG、AI Agents and Agentic Systems、Fine-Tuning and Model Adaptation、Vector Databases and Embeddings、AI System Design、LLMOps and Production AI、Evaluation and Testing、AI Safety・Ethics、Multimodal AI、AI Infrastructure and Scalability、Coding and Practical Implementation、Behavioral and Scenario-Based Questions が並びます。LLM 基礎では Transformer、Attention の Q/K/V、$\sqrt{d_k}$ スケーリング、KV cache、RoPE、RMSNorm、MoE、Flash Attention、GQA などを扱います。PPO、DPO、GRPO といったアラインメント手法や、Diffusion Language Models、LLM Watermarking のような比較的新しい話題もあります。

回答の形式と構造

各質問は箇条書きで、その下に Answer として外部リソースへのリンクが付く構成です。リンク先の大半は outcomeschool.com のブログ記事で、ほかに YouTube 動画「AI Engineering Explained: LLM, RAG, MCP, Agent, Fine-Tuning, Quantization」、Substack、X、LinkedIn の投稿があります。つまり README 単体で答えを読めるわけではなく、質問リストと学習リソースへの索引として機能します。また「Explain WordPiece and SentencePiece.」や Chinchilla のスケーリング則など、回答リンクがまだ付いていない質問も少なくありません。

実務寄りのシナリオ質問

大きな特徴は「Your RAG system is hallucinating despite having the right context. How do you fix it?」「Your AI agent deleted a production database. How do you prevent irreversible actions?」のような、実運用で起こるトラブルを想定した質問が多いことです。p99 レイテンシの悪化、プロバイダの障害やレートリミット、埋め込みモデル更新による検索品質の低下、量子化による精度劣化、エージェントの無限ループやトークン浪費などが取り上げられています。システム設計では Real-Time Voice AI Agent、vLLM-as-a-Service、AI Coding Agent、1000万文書規模のエンタープライズ RAG などのお題が並びます。

使い方

インストールやビルドは不要で、GitHub 上で README を読むだけです。カテゴリごとに質問を見て自分で答えを考え、そのあとリンク先の記事や動画で確認する、という自習の流れが想定されます。回答リンクのない質問は、自分で調べて答えをまとめる練習にも使えます。Apache-2.0 ライセンスなので、ライセンス条件に従えば社内勉強会などで再利用することもできますが、リンク先コンテンツの利用条件は README からは不明です。

注目ポイント・注意点

スター数は 3500 超、フォークは 600 超と注目を集めています。MCP、Agent Skills、Harness Engineering、Context Engineering、context compaction、Claude Code や Cursor の仕組みなど、エージェント時代のトピックをいち早く取り込んでいる点は価値があります。一方で、回答がほぼ作者が運営する Outcome School のコンテンツへのリンクに集約されているため、情報源が偏る点や、外部サイトの状態に依存する点には注意が必要です。提供された README は途中で切れており、Multimodal AI 以降のカテゴリの詳細は確認できませんでした。

技術構成: Markdown
向いている人: AI Engineer、LLM Engineer、AI エージェント開発者、MLOps/LLMOps エンジニアなどの職種の面接を控えた人に向いています。LLM、RAG、エージェント、推論最適化、評価、安全性の知識を体系的に棚卸ししたい現役エンジニアや、生成 AI 分野の学習ロードマップを探している学生にも、質問集兼リンク集として使えます。

テーマ 14. aiming-lab/MetaClaw

https://github.com/aiming-lab/MetaClaw

★ 3,465 ／ フォーク 453 ／ 言語 Python ／ 作成 2026-03-09 ／ MIT
該当テーマ: LLM fine-tuning、LoRA LLM

普段の会話から学習し進化するパーソナルAIエージェント向けのプロキシ型メタ学習フレームワーク

概要

MetaClawは、OpenClawなどのパーソナルエージェントとLLM APIの間にプロキシとして入り、日々の会話を学習シグナルに変えてエージェントを継続的に改善する仕組みです。会話ごとにスキル（Markdownの指示）を注入・自動要約し、オプションでクラウド上のLoRA強化学習（GRPO）による重み更新も行います。GPUクラスタは不要で、OpenAI互換APIとTinker互換バックエンドだけで動く点が特徴です。さらにRL更新を睡眠時間やアイドル時間に回すスケジューラや、セッションをまたぐ長期記憶レイヤーも備えています。

要点

- metaclaw setupとmetaclaw startの2コマンドで導入
- skills_only・rl・autoの3モード、既定はauto
- スキル注入と会話後の自動スキル要約で進化
- Tinker/MinT/WeaverでLoRA強化学習、GPU不要
- OpenClaw等8種以上のエージェントを自動設定

何をするものか

MetaClawは「in the wild（実運用の場）」でメタ学習し進化するエージェントを掲げるプロジェクトです。ユーザーは普段どおりエージェントと会話するだけでよく、MetaClawがその会話を傍受して学習シグナルに変換します。オフライン学習だけに頼らず、実際の利用を通じて継続的に改善していくことが狙いです。技術レポート「MetaClaw: Just Talk -- An Agent That Meta-Learns and Evolves in the Wild」がarXiv（2603.17187）で公開されており、READMEによればHuggingFace Daily Papersで1位になったとされています。ライセンスはMITです。

主な機能

動作モードは3種類です。skills_onlyはLLM APIをプロキシし、毎ターン関連スキルを注入し、セッション後に新しいスキルを自動要約する最軽量モードで、GPUもTinkerも不要です。rlはこれに加えてGRPOによるRL学習を行い、バッチが埋まり次第即時学習します。教師モデルから蒸留するOPD（On-Policy Distillation）もオプションで使えます。既定のautoはRLにスケジューラを加え、重み更新を睡眠時間・キーボード無操作時間・Google Calendarの会議中といったユーザー不在時間に回します。v0.4.0以降はepisodic、semantic、preference、project_state、working_summaryの5種類を扱う長期記憶レイヤーも追加されています。

仕組み・アーキテクチャ

MetaClawはローカルのプロキシ（既定ポート30000）として動き、OpenAI互換の/v1エンドポイントとAnthropic互換の/v1/messagesエンドポイントを公開します。エージェントからのリクエストに、~/.metaclaw/skills/のSKILL.mdから検索したスキル（template または embedding検索）と記憶を注入します。RLモードでは各ターンをトークナイズして学習サンプルとし、判定用LLM（PRM）が非同期に応答をスコアリング、Tinker互換バックエンドがLoRA学習を行い重みをホットスワップします。失敗エピソードからはevolver LLMが新スキルを抽出します。サービング・報酬計算・学習は完全に分離された非同期設計です。また各サンプルにskill_generationを付け、スキル進化時にRLバッファをフラッシュするMAML風のsupport/query分離で古い報酬信号の混入を防いでいます。

技術構成と対応エージェント

主要言語はPythonで、pipでインストールします。追加依存はextrasで分かれており、rlでtorch、transformers、tinker、evolveでOpenAI互換LLMによるスキル進化、schedulerでGoogle Calendar連携が入ります。MinTはmindlab-toolkit、Weaverはnex-weaverを別途導入します。LLMプロバイダはKimi、Qwen、OpenAI、MiniMax、Novita、OpenRouter、Volcano Engineなどに加え、OAuthトークン経由でAnthropic（Claude Code）、OpenAI Codex、Gemini CLIも利用可能です。claw_type設定によりOpenClaw、CoPaw、IronClaw、PicoClaw、ZeroClaw、NanoClaw、NemoClaw、Hermes Agentの設定ファイルを自動で書き換え再起動します。WeChat連携用プラグインの自動導入にも対応しています。

使い方・導入方法

pip install -e . でskills_only向けの軽量インストール、フル機能ならpip install -e ".[rl,evolve,scheduler]" が推奨です。OpenClaw利用者はリリースのmetaclaw-plugin.zipをextensionsに展開し、openclaw plugins enable metaclaw-openclawで有効化する手順も用意されています。その後metaclaw setupの対話ウィザードでエージェント種別、認証方式、LLMプロバイダ、モデルIDを選び、metaclaw startで起動します。設定は~/.metaclaw/config.yamlに保存され、metaclaw config KEY VALUEで変更可能です。metaclaw status、metaclaw stop、metaclaw uninstallなどの管理コマンドもあります。内蔵の40以上のスキルバンクはmemory_data/skillsからコピーして事前ロードできます。

注目ポイント・注意点

GPUを持たずにAPIだけで継続学習を体験できる点、RLの重み更新でエージェントが数分停止する問題をスケジューラで回避している点が大きな工夫です。評価用にMetaClaw-Bench（30日版と12日版）が同梱されており、複数日の対話履歴からの適応力を測れます。一方で、RLモードではTinker等の外部学習サービスやPRM用のAPIキーが必要で、利用料金などのコストはREADMEからは不明です。会話内容が外部の判定LLMや学習バックエンドに送られる構成になるため、プライバシー面の扱いは利用者側で確認が必要です。提示されたファイル一覧は主にベンチマークデータで、本体コードの詳細構成はREADMEからは不明です。

技術構成: Python、PyTorch、transformers、Tinker、MinT (mindlab-toolkit)、Weaver (nex-weaver)、LoRA、GRPO、OpenAI互換API、Anthropic Messages API、Google Calendar API、vLLM / SGLang（OPD教師モデル用）
向いている人: OpenClawなどのパーソナルAIエージェントを日常的に使い、使うほど賢くなるエージェントを試した

い開発者や、GPUなしでオンライン強化学習・継続学習・メタ学習の実装を研究・検証したい研究者や学生に向いています。

テーマ 15. Ch1rpys/Mirrai

https://github.com/Ch1rpys/Mirrai

★ 1,268 ／ フォーク 8 ／ 言語 TypeScript ／ 作成 2026-05-05 ／ MIT
該当テーマ: LLM distillation

実際のチャット履歴から相手の人格を再構築し、最後は「卒業」で別れまで支える自前ホスト型AIコンパニオン

概要

Mirraiは、WeChatなどの実際のチャット履歴をアップロードすると、多段階の人格蒸留パイプラインで相手の口調や癖、感情の動きを再現した「デジタル分身」を作るオープンソースのAIコンパニオンです。データはローカル保存が基本で、OpenAI、Claude、DeepSeek、Ollamaなど12のLLMプロバイダーを切り替えて使えます。最大の特徴は、利用者を引き留めることではなく別れを手助けする「卒業機制」です。成功指標を継続率ではなく卒業率に置いています。React 19とtRPCで型安全に組まれたフルスタックTypeScriptアプリで、Docker、macOS、Windows向けのインストーラーも用意されています。

要点

- チャット履歴から7層構造のペルソナを多段階で蒸留
- 6状態の感情ステートマシンと親密度システム
- 別れを支える卒業機制と感情安全レイヤーを搭載
- 12のLLMプロバイダー対応、Ollamaで完全ローカル推論も可能
- WeChat公式ClawBotと飛書(Lark)の通道に対応

何をするものか

Mirraiは「不在身边的时候，TA 还在这里（そばにいない時も、TAはここにいる）」をコンセプトにしたAIコンパニオンです。恋人や友人、家族とのチャット履歴、写真、動画をアップロードすると、その人の話し方、口癖、愛着スタイル、喧嘩の仕方などを抽出します。そこから、その人らしく返信するデジタル分身を作ります。開発者はNLP専攻の学生で、身近な人を失う不安から作り始めたと創作手記に書かれています。一方で、README は「本物の関係の代わりではない」と明記しており、言えなかった言葉を言い終えるための場所として位置づけています。対象者本人の同意を得ること、AIであると透明に示すことなど、倫理面の注意書きも詳しく載っています。

主な機能

ペルソナ蒸留には「快速 / 标准 / 深度 / 同事 / 名人研究」の5モードがあります。生成されるペルソナは、性格特質、話し方、口癖、愛着タイプ、愛の言語、喧嘩スタイル、心が動く瞬間の7層です。感情状態は温柔、俏皮、思念、忧郁、开心、疏离の6種類で、会話に応じて切り替わります。判定方法は EMOTION_ENGINE で llm と heuristic から選べます。 記憶まわりでは、一定量の会話から記憶を自動抽出し、関連度でスコアリングして呼び出します。どの記憶を呼び出したかは返信ごとに表示されます。ほかに、遅れて返事が届く「信」、ユーザー自身の気分記録、関係イベントの記録、タイムゾーン対応、SSEによるストリーミング応答、Edge TTSによる読み上げ、SillyTavern角色卡(PNG v2)のインポートとエクスポート、PWAにも対応しています。

卒業機制と安全設計

親密度は会話とともに5段階で上がっていきます。ユーザーの感情が穏やかになってきたと判断されると、システムが「卒業」を一度だけ提案します。卒業すると、分身の口調で書かれたお別れの手紙が生成され、分身は休眠に入ります。休眠中の分身はいつでも起こせます。提案方法はシステム提案、手動のみ、オフから選べ、分身ごとにも設定できます。「先缓一缓（少し休む）」という一時停止状態もあります。 感情安全レイヤーは、自傷を示す表現を検知するとローカルの相談窓口を案内します。このとき外部への通報やネット送信はしません。長時間の会話や1日の頻度の多さ、依存傾向の上昇も検知して、気遣いのメッセージを出します。卒業時には、ユーザーがここでしか言えなかった言葉をまとめて提示します。ただし、システムがそれを本人へ代わりに送ることはありません。

仕組み・アーキテクチャ

クライアントはReact 19で、WouterによるルーティングにRadix UIとTailwind CSSを組み合わせています。サーバーはExpressで、tRPC(HTTP Batch + SuperJSON)を通じて通信します。Drizzle SchemaからDB、tRPC Router、React Query Hook、UIまで型推論がつながっています。 サーバー側の主なディレクトリは次のとおりです。chat は対話パイプラインとSSE、channels は微信と飛書のアダプター、safety は安全レイヤー、llm はプロバイダー抽象化層、scheduler はリマインダーを担当します。人格蒸留はPythonのサブプロセス(skill-engine)で動きます。レート制限とスケジューラーにはRedisを使いますが、なくても動作し、その場合はプロセス内カウントに戻ります。 再現度の評価もあります。pnpm eval は、実際の履歴から隠しておいた一文と分身の発話を比べ、三文字一致率や尤度差などを測ります。これは手元のマシンで実行します。CIでは、確定的なチェックを行う pnpm fidelity:gate が動きます。

使い方・導入方法

推奨はDockerです。git clone の後に cp .env.example .env を実行し、.env に JWT_SECRET と少なくとも1つのLLM APIキーを書いてから docker compose up -d で起動します。起動後は http://localhost:3000 にアクセスします。PostgreSQLとRedisは同梱されています。 ソースから動かす場合は、Node.js 24+、pnpm、PostgreSQL 14+ が必要です。蒸留エンジンを使うならPython 3.9+ も用意します。pnpm install、pnpm db:push、pnpm dev の順に実行します。既存環境のアップグレードには pnpm db:migrate を使います。macOS(Apple Silicon)向けのdmgと、Windows x64向けのexeインストーラーもReleasesで配布されています。どちらもNode.jsとPostgreSQLを内蔵しています。OpenAI互換のサービスは baseUrl と model を指定するだけで追加できます。

注目ポイント・注意点

WeChat連携は、Wechatyのような非公式プロトコルではなく、公式の「ClawBot」プラグイン(腾讯 iLink)を使います。そのためアカウント凍結のリスクはないと説明されています。ただし、WeChat 8.0.70以上が必要で、機能は段階的に公開中のため、すべてのユーザーが使えるわけではありません。飛書通道は企業の自社開発アプリに限られ、管理者の承認も必要です。 分身の質は素材の量と使うLLMに左右されます。README は100件以上のメッセージと、GPT-4oまたはClaudeの使用を勧めています。テストファイルは122個あり、CIのほかCodeQLやSemgrepも設定されています。未完了の項目は docs/open-items.md にまとめられており、全体の国際化、プラグインシステム、グループチャットは今後の予定です。実在の人物を扱うので、同意とプライバシーには十分注意してください。

技術構成: TypeScript、React 19、Vite、Tailwind CSS、Radix UI、Wouter、tRPC、TanStack React Query、Express、PostgreSQL、Drizzle ORM、Redis、jose(JWT)、Zod、Framer Motion、Vitest、Python(skill-engine)、edge-tts、@larksuiteoapi/node-sdk、Docker、Ollama
向いている人: 大切な人とのチャット履歴をもとに、言えなかった言葉に向き合いたい個人に向いています。データを外に出さず自前環境でAIコンパニオンを動かしたい人や、Ollamaなどでローカル推論をしたい人にも合います。ペルソナ蒸留、感情ステートマシン、記憶の呼び出し、再現度評価の実装を学びたいLLMアプリ開発者や学生、tRPCとDrizzleによる型安全なフルスタック構成の実例を探しているTypeScriptエンジニアにも参考になります。

テーマ 16. lasgroup/SDPO

https://github.com/lasgroup/SDPO

★ 1,137 ／ フォーク 130 ／ 言語 Python ／ 作成 2026-01-24 ／ Apache-2.0
該当テーマ: LLM distillation

LLMの強化学習に自己蒸留を組み合わせ、豊富なフィードバックを密な学習信号に変えるSDPOの公式実装

概要

SDPO（Self-Distilled Policy Optimization）は、コードや数学などの検証可能な領域で行うLLMの強化学習（RLVR）向けの手法です。RLVRは試行ごとにスカラー報酬しか得られず、どのトークンが良かったのかを割り当てにくいという課題があります。SDPOは実行時エラーなどのテキストフィードバックや自身の高報酬な解答を条件として与えた現在のモデルを「自己教師」とみなし、その次トークン予測を方策へ蒸留します。外部の教師モデルや報酬モデルを使わずに密な信号を得られる点が新しく、論文（arXiv:2601.20802）の再現コードとしてverlをベースに公開されています。

要点

- スカラー報酬の信用割り当て問題を自己蒸留で緩和する
- 外部教師モデルや報酬モデルは不要で、自分自身を教師にする
- リッチなフィードバックがない環境でもGRPOとの比較結果を報告
- 推論時の自己蒸留（test-time self-distillation）にも対応
- verlをベースに、loss_modeをsdpoに切り替えるだけで有効化できる

何をするものか

LLMの事後学習で広く使われるRLVRでは、1回の試行につき正解か不正解かのようなスカラー報酬しか学習に使えず、長い出力のどこが悪かったのかを特定しにくいという信用割り当てのボトルネックがあります。一方で、多くの検証可能な環境は実行時エラーや判定器の評価など、失敗の理由を説明するテキストを返します。著者らはこれをReinforcement Learning with Rich Feedback（RLRF）として定式化し、その情報を活かす手法としてSDPOを提案しています。本リポジトリは論文の実験を再現するための実装です。

仕組み・アーキテクチャ

SDPOでは、元のプロンプトに成功した過去の解答や環境からのフィードバックを付け加えた「再プロンプト」を作ります。この再プロンプトを条件とした現在のモデルを自己教師とし、その次トークン分布を元の方策に蒸留します。モデルが文脈内で自分の誤りを振り返って見つける能力を利用する発想です。蒸留にはロジット全体のKL（alphaで順方向KL、逆方向KL、JSDを切り替え）を使え、top-kロジットに残りの確率をまとめるtailバケットを足す近似も選べます。教師はEMAまたはtrust-regionで正則化され、重要度サンプリング比のクリップも設定できます。リッチなフィードバックがない場合でも、高報酬のロールアウトを暗黙のフィードバックとして再利用します。

主な結果

Olmo3-7B-InstructのChemistryタスクなどで、GRPOと比べた学習推移を報告しています。推論系ベンチマークでは、4基のNVIDIA GH200を載せたノードで1時間と5時間の学習時点におけるavg@16を比較しています。LiveCodeBench v6を使ったリッチフィードバックの設定では、信用割り当てが細かいほど性能が高い（logit、token、sequenceの順）と述べています。また、学習が進むと自己教師自体も改善し、最終的な生徒モデルが初期の教師を大きく上回るとしています。さらに推論時の自己蒸留では、ベースモデルやマルチターン対話では解けない難しいコーディング問題を解けると報告しています。

技術構成

実装は強化学習フレームワークverlをベースにしており、リポジトリの大部分はverl由来のdocs、examples、dockerなどです。SDPO固有の設定はactor.policy_lossとactor.self_distillationに追加されています。言語はPython 3.12で、PyTorch（Ampere/Hopperでは2.5.1+cu124、Blackwellでは2.7.0+cu128）とFlash Attention 2を使います。高スループット推論用にはvLLMやSGLangを任意で導入でき、データはparquet形式に前処理します。実験ログはWeights & Biasesで公開されています。データ周りにはSciKnowEval（生物・化学・材料・物理）、tooluse、LiveCodeBench、HumanEval、MBPP、Codeforces、TACOなどの読み込みスクリプトが含まれています。

使い方・導入方法

動作環境はLinuxとNVIDIA GPUです。GH200（aarch64、CUDA 13.1）向けにはNGC vLLMコンテナをベースにしたDockerfile.gh200があり、podmanでビルドしてenrootで取り込めます。ローカルに入れる場合は、PyTorchを入れた後にpip install -r requirements.txt、pip install -e .、pip install flash-attn --no-build-isolationの順で実行します。データは前処理済みのものがdatasetsにあり、data/preprocess.pyでparquetに変換します。次にverl/trainer/config/user.yamlのパスを自分の環境に合わせ、experiments/generalization/run_sdpo_all.shやexperiments/rich_feedback/run_sdpo.shを実行すると結果を再現できます。GRPOのベースラインスクリプトも用意されています。

注目ポイント・注意点

既存のverlの設定にloss_mode: sdpoを加えるだけで試せるため、GRPOとの比較実験を組みやすい構成です。再プロンプトのテンプレート、thinkタグの除去、自分自身の成功例を手本に使わないオプションなど、細かい制御も用意されています。ただし、Blackwell世代（RTX 50系、B100/B200）への対応は十分にテストされていないと明記されています。trust-region正則化はuse_fused_kernels = Falseが必要です。論文の実験はGH200を4基使い1回あたり約6時間かかっており、計算資源の要求は大きめです。小規模なGPUでどこまで動くかはREADMEからは不明です。

技術構成: Python 3.12、PyTorch、verl、Flash Attention 2、vLLM、SGLang、Ray、Docker / Podman / enroot、CUDA、Weights & Biases、parquet
向いている人: LLMの強化学習（RLVR、GRPOなど）を研究するエンジニアや研究者、学生に向いています。特に、コード生成や科学・数学の推論タスクで、エラーメッセージなどのフィードバックを学習に活かしたい人や、SDPO論文を再現・拡張したい人に適しています。マルチGPU環境とverlの基本的な知識があると扱いやすいでしょう。

テーマ 17. CodeBendKit/codeseek

https://github.com/CodeBendKit/codeseek

★ 761 ／ フォーク 44 ／ 言語 Rust ／ 作成 2026-06-03 ／ MIT
該当テーマ: reranker

AIコーディングエージェント向けに、コールグラフとハイブリッド意味検索を提供するRust製CLI

概要

CodeSeekは、Tree-sitterでソースコードを解析してコールグラフと検索インデックスを構築し、ターミナルやClaude Code、Codex CLIから関数の呼び出し関係や自然言語によるコード検索を行えるようにするツールです。検索はDense検索(LanceDB)、Sparse検索(Tantivy BM25)、グラフ検索をRRFで統合し、Cross-Encoder Rerankerで並べ替える構成になっています。codeseek installでMCPツールとして登録でき、AIエージェントが大規模なコードベースを効率よく探索できるようになります。デーモンやHTTPサーバーを常駐させず、コマンドごとに独立したプロセスで動く点も特徴です。

要点

- Rust、Python、JS、TS、Go、C/C++、Javaの7言語に対応
- Dense+BM25+グラフ検索をRRFで統合し、Rerankerで精度を高める
- Claude Code/Codex CLIにMCPツールとしてワンコマンドで登録
- MD5による差分インデックスとgit hookによる自動更新
- npm、Homebrew、ソースビルドの3通りで導入できる

何をするものか

CodeSeekはClaude Code向けのコードインテリジェンスCLIです。ASTに基づくコールグラフ解析と意味検索をターミナルから実行できます。codeseek callersやcodeseek calleesで、ある関数の呼び出し元や呼び出し先をたどれます。codeseek search 'how the code embedding work' のように自然言語で問い合わせると、関連するシンボルがスコア付きで返ります。結果はMCPツール(codeseek_search、codeseek_callers、codeseek_callees、codeseek_callgraph、codeseek_status)としてAIエージェントにも渡せるため、エージェントがコードベースの構造を把握する手段になります。

主な機能

主なコマンドは次のとおりです。codeseek initはインデックスを構築し、初回は全体、2回目以降は差分だけを処理します。codeseek searchはシンボル検索、codeseek callgraphは深さを指定して双方向にコールグラフを引きます。codeseek listはインデックス済みプロジェクトの一覧、codeseek statusは関数数などの統計を表示します。codeseek install / uninstallはMCP連携の登録と解除、codeseek install-hooksはpost-commit/post-mergeでの自動再インデックスを設定します。codeseek serve --mcpはstdio JSON-RPCのMCPサーバーを起動します。問い合わせ系のコマンドはすべて--jsonで機械可読な出力に切り替えられます。

仕組み・アーキテクチャ

インデックス構築は次の流れで進みます。まずTree-sitterでASTを解析し、関数・クラス・メソッドを抽出します。次にAPI経由でまとめて埋め込みを生成し(1回の呼び出しで20件、SQLiteにキャッシュ)、ベクトルをLanceDBに、BM25索引をTantivyに保存します。コールグラフはPetCodeGraphとしてシリアライズし、すべてを~/.codeseek/<project_hash>/に置きます。検索時はDense検索、Sparse検索、グラフ検索の結果をReciprocal Rank Fusionで上位20件に絞り、Qwen3-Reranker-4BのCross-Encoderで再スコアリングして上位5件を返します。埋め込みやRerankerが使えない場合は、APIを必要としないグラフベースの名前検索に切り替わります。

技術構成

本体はrust-core配下のRustで書かれています。READMEにはTree-sitter、Petgraph、LanceDB、Tantivy、Tokio、Clapを使っていると記載があります。ソースはcodegraph(言語別パーサー、skeletonizer)、services(embedding、hybrid_search、reranker、callgraph)、storage(petgraph_storage、tantivy_index、incremental)、mcp(server、tools)に分かれています。npmパッケージはTypeScript製の薄いラッパーで、初回設定ウィザードの実行、プラットフォームに合ったバイナリのGitHub Releasesからの取得、コマンドの転送を担います。Node.js 20以上が必要で、ソースからビルドする場合はprotocも必要です。

使い方・導入方法

もっとも手軽なのはnpm install -g codeseekです。初回にcodeseekを実行すると対話ウィザードが起動し、埋め込みAPIのトークン、モデル、ベースURLを設定できます。続いてプロジェクトでcodeseek initを実行し、codeseek installでClaude Code(~/.claude.jsonまたは./.mcp.json)やCodex CLI(~/.codex/config.toml)に登録します。Homebrewのtapや./build.sh --releaseによるソースビルドにも対応しています。設定は~/.codeseek/config.jsonにあり、BM25やベクトル検索のtop_k、rrf_k、Rerankerの有効化なども細かく調整できます。

注目ポイント・注意点

既定の設定例では、埋め込みにQwen/Qwen3-Embedding-4B、再ランキングにQwen/Qwen3-Reranker-4Bを使い、OpenAI互換API(例: SiliconFlow)を呼び出す構成です。そのため意味検索にはAPIトークンが必要で、コードの一部が外部APIへ送信される点に注意が必要です。Rerankerの処理には1〜2秒ほどかかると記載されています。READMEに記載されている対応プラットフォームはmacOS(arm64/x64)とLinux x64で、Windowsへの対応はREADMEからは不明です。ローカル埋め込みモデルを利用できるかどうかもREADMEからは不明です。

技術構成: Rust、TypeScript、Node.js、Tree-sitter、Petgraph、LanceDB、Tantivy、Tokio、Clap、SQLite、MCP(JSON-RPC)、Qwen3-Embedding-4B、Qwen3-Reranker-4B
向いている人: Claude CodeやCodex CLIなどのAIコーディングエージェントに、大規模なコードベースの呼び出し関係や意味検索を使わせたい開発者に向いています。Rust、Python、Go、Javaなど複数言語が混在するリポジトリをターミナルから素早く調べたいエンジニアにも役立ちます。ハイブリッド検索とRerankerの実装例を学びたい人にとっても参考になります。

テーマ 18. MoMoM101/RAG-ReActAgent

https://github.com/MoMoM101/RAG-ReActAgent

★ 208 ／ フォーク 0 ／ 言語 Python ／ 作成 2026-07-23 ／ MIT

該当テーマ: reranker

Qdrant と BM25 のハイブリッド検索、自作 ReAct ループ、引用検証を組み合わせたローカル優先の文書ナレッジベース QA システム

概要

RAG Agent は、アップロードした PDF や Word などの文書に自然言語で質問し、出典付きで回答を得るためのローカル優先の RAG システムです。「文書を入れて LLM を一度呼ぶだけ」の実装ではなく、入庫タスクの永続化と再起動後の復旧、Qdrant と SQLite BM25 のハイブリッド検索、自作 ReAct エージェント、回答後の引用・数値検証、JWT 認証、バックアップ復元、評価ゲートまでを一つの工学パイプラインとして揃えています。根拠のない回答を減らす回答検証と、合成データによる再現可能な評価を公開している点が特徴です。現在のバージョンは v0.2.1-beta です。

要点

- Qdrant と SQLite BM25 を重み付き RRF で融合し、Reranker も任意で使える
- 自作 ReAct ループで、search_docs や web_search などのツールを自律的に選んで呼び出す
- 回答後に引用の実在性や数値の一致を検証し、根拠がなければ回答を拒否する
- 入庫タスクは永続化され、リトライ・デッドレター・再起動後の復旧に対応
- 合成ベンチマーク93問で忠実度100%と公表しているが、held-out テストではない

何をするものか

ユーザーが文書をアップロードすると、システムが解析・チャンク分割・Embedding・索引化を行い、その後は自然言語の質問に対して意図分類、クエリの曖昧さ解消、ハイブリッド検索、ツール呼び出し、根拠の整理、ストリーミング回答、引用検証までを実行します。対応形式は .pdf、.docx、.txt、.md、.csv、.xlsx の6種類です。想定用途は、社内規程・操作マニュアル・技術文書の QA、エラーコードや型番の正確な検索、複数文書の比較などです。アップロードファイル、SQLite、Qdrant、BM25、モデルキャッシュは既定でローカルに保存され、サーバーも既定では 127.0.0.1 のみで待ち受けます。データを手元に置いたまま運用できる設計です。

主な機能

検索は二系統を並行して実行します。Qdrant による意味検索と、SQLite BM25 によるキーワード検索です。後者は中国語の分かち書きとコード文字列の保持に対応しています。二つの結果は重み付き RRF で融合し、重複除去、チャンク品質フィルタ、任意の Cross-Encoder 精排を通します。エージェントの内蔵ツールは search_docs、web_search、calculator、list_documents、get_document_info、recall_memory の6つです。回答には SSE ストリーミング、ツール呼び出しカード、出典カード、行内引用が付きます。このほか、会話管理、長期メモリ（好み・決定・プロジェクト事実の抽出）、Prometheus 指標、監査ログ、バックアップと復元も備えています。

仕組み・アーキテクチャ

構成は React 19 のフロントエンドと FastAPI のバックエンドで、両者は JWT と SSE で通信します。ReAct ループでは、モデルが観察、判断、呼び出し、評価を繰り返します。知識ベースだけでは足りないと判断すると web_search に切り替えます。同じツールを3回以上続けて呼ぶと強制的に収束させ、無限ループを防ぎます。ハードな制約として残しているのは、ループ上限、コンテキスト裁断、後検証の3層です。後検証では、存在しない引用（例: [S99]）の修正、数値の一致確認、必須引用の漏れチェックを行い、決定的な修復か、回数を制限した LLM 修復を実行します。知識ベースの引用は S、Web 検索の引用は WS という接頭辞で区別します。

技術構成

バックエンドは Python 3.12 と FastAPI です。データベース移行には Alembic を使い、移行前に SQLite のスナップショットを取ります。ディレクトリは agent、rag、embedding、llm、textdb、vectordb、reranker、ocr、memory、worker に分かれています。LLM と Embedding は OpenAI 互換 API を抽象化して扱い、既定値は gpt-4o と text-embedding-3-small です。Reranker の既定モデルは BAAI/bge-reranker-v2-m3 です。OCR はファイル名に paddle_ocr.py があり、PaddleOCR 系と見られます。フロントエンドは React 19、Zustand、Playwright、Vitest、Oxlint を使っています。トピックには weaviate が含まれていますが、README とファイル構成から確認できるベクトル DB は Qdrant です。

使い方・導入方法

最も手軽なのは GHCR イメージを使う方法です。リポジトリを clone し、backend/.env.example を backend/.env にコピーして、docker compose --env-file backend/.env -f docker-compose.ghcr.yml up -d を実行します。起動後は http://localhost:5173 を開きます。初回の管理者アカウントは admin / RAGAgent2026! で、ログイン直後のパスワード変更が推奨されています。ローカル開発の場合は、仮想環境で pip install -r backend/requirements.txt を実行し、frontend で npm install を実行してから、python main.py で前後端をまとめて起動します。OCR と Reranker は追加の requirements ファイルを入れて有効化します。LLM と Embedding のキーは、起動後に設定画面から入力できます。

注目ポイント・注意点

評価データ、qrels、再現コマンド、失敗事例が公開されており、引用を強制しない対照群と比べて忠実度が 72.35% から 100% に上がったと報告されています。ただし作者自身が、合成データかつ開発用サブセットを含むため実務データにそのまま当てはまらないと明記しています。Reranker による品質向上が見られなかったことも正直に記載されています。外部 LLM を使う場合は、質問や文書チャンクが外部へ送信される点に注意が必要です。公開環境では HTTPS、AUTH_COOKIE_SECURE=true、固定の JWT_SECRET の設定が求められます。また OCR_ENABLED の既定値は、README 内で false とする記述と true とする記述が混在しています。開発には Claude Code や Codex などの AI 支援を使ったと開示されています。

技術構成: Python 3.12、FastAPI、uvicorn、React 19、TypeScript、Zustand、Qdrant、SQLite BM25、Alembic、JWT、SSE、OpenAI 互換 API、Cross-Encoder Reranker (BAAI/bge-reranker-v2-m3)、PaddleOCR、Prometheus、pytest、Ruff、MyPy、Vitest、Playwright、Oxlint、Docker Compose
向いている人: 社内文書や技術資料を、データを手元に置いたまま出典付きで検索・QA したいチームや個人に向いて

います。ハイブリッド検索、ReAct エージェント、引用検証、評価ゲートなど、本番を意識した RAG の実装例を学びたいエンジニアや学生にも参考になります。

テーマ 19. zhengkid/AutoTTS

https://github.com/zhengkid/AutoTTS

★ 178 ／ フォーク 16 ／ 言語 Python ／ 作成 2026-05-10
該当テーマ: test-time scaling LLM

LLMのテスト時スケーリング（TTS）戦略を、コーディングエージェントがオフライン再生環境上で自動探索する研究「AutoTTS」の公式実装

概要

AutoTTS は、複数の推論ブランチをいつ増やし、いつ打ち切り、いつ回答するかといったテスト時スケーリングの制御戦略を、人手のヒューリスティクス設計ではなく、環境駆動の自動探索で発見する研究の公式リポジトリです。人間は事前に推論トレースをキャッシュしたオフライン再生環境だけを用意し、Claude Code などのコーディングエージェントがコードで定義されたコントローラを繰り返し提案・改良します。勾配更新は行わず、探索中の評価はキャッシュの再生のみで LLM 呼び出しは0回、1回の探索は約39.9ドル・160分と報告されています。発見されたコントローラ CMC は、SC@64 と同等の held-out 精度を保ちつつトークンを約69.5%削減したとされています。

要点

- TTS戦略をコード編集による自動探索で発見し、勾配更新は行わない
- 探索中の評価はキャッシュ再生のみでLLM呼び出し0回
- 発見されたCMCはSC@64比で約69.5%のトークン削減
- 単一のβでコストと精度のトレードオフを調整できる
- 評価のみなら numpy・pandas・tqdm だけで再現可能

何をするものか

推論時に計算資源を追加して精度を上げるテスト時スケーリングでは、Self-Consistency（SC@64）や ASC、ESC のように、複数の推論パスを並列に生成して多数決をとる手法が使われます。ただし、いつ止めるか、何本に広げるかといった制御は人手で設計されてきました。AutoTTS はこの設計を、有限の予算をブランチに割り当てる逐次意思決定問題として定式化します。行動は BRANCH・CONTINUE・PROBE・PRUNE・ANSWER の5種類です。そのうえで、制御ポリシーのプログラムそのものを LLM エージェントに探索させます。AIME24 で最適化し、AIME25 と HMMT25 を held-out として、Qwen3 の 0.6B・1.7B・4B・8B の4スケールで評価しています。

仕組み・アーキテクチャ

まず (モデル, ベンチマーク) ごとに一度だけ再生環境を構築します。各問題についてバックボーンから N 本の独立した推論トレースを生成し、Δ トークンの固定長セグメントに分割します。各プレフィックスでの probe 応答もあわせて表として保存します。以降の環境遷移はこの凍結済みテーブルを参照するだけなので、新たなデコードは発生しません。外側のループでは、エージェントが method.py の OptimalController を書き換えます。候補は β をスイープしながら再生評価され、スケーリング曲線と行動ごとの実行トレースが次ラウンド

の履歴に追加されます。β パラメータ化とは、全内部ハイパーパラメータを単一スカラー β から決定的・単調に導く方式で、探索セットだけに過適合した閾値を避ける狙いがあります。

発見されたコントローラ CMC

Confidence Momentum Controller（CMC）は、プール内の完了回答から計算した信頼度の指数移動平均（EMA）を追跡します。EMA が閾値以上で、かつ傾向が非負のときだけ停止するため、一時的な信頼度のスパイクで早期停止することを防げます。信頼度の伸びが弱い、あるいは低下しているときにはブランチを追加して探索を広げます。多数派の回答と一致するブランチには追加の probe ステップを与えます。長期間逸脱し続けたブランチは放棄しますが、生存ブランチは最低2本残します。n_init、max_branch_use、conf_thresh などは β の線形関数として定義されています。β=0 では低コスト寄り、β=1 では精度寄りの挙動になります。全ソースは README に掲載されています。

技術構成

Python 3.12 の Conda 環境が前提です。評価部分は numpy、pandas、tqdm のみで動きます。探索ワークフローは run_workflow.sh から workflow_propose_critic.py を起動する構成で、提案器として claude_proposer.py（claude-agent-sdk==0.1.58 と Claude Code CLI を使用）と codex_proposer.py が用意されています。API は OpenRouter 経由で設定する例が示され、claude-sonnet-4.6 や claude-opus-4.6 などのモデル名が指定されています。再生データは JSON 形式です。探索用の environment/ と、提案器に見せない test_environment/ に分離されています。ベースラインとして ASCMethod、ESCMethod、Parallel_Probe が同梱されています。

使い方・導入方法

再現方法は2通りあります。Way A は同梱のコントローラを評価するだけの方法で、API や Claude ツールは不要です。cd efficient_reasoning_controller を実行した後、python eval/eval.py を実行すると、eval/test_results/ 以下に CSV とトレースの JSONL が出力されます。過去ラウンドのスナップショットを評価したい場合は、logs/search_history 内の method.py を eval/method.py にコピーします。Way B は探索を自分で回す方法です。Node.js 21、Claude Code、API キーを設定し、huggingface-cli download AutoTTS/history で履歴データを取得してから、bash controller_search/run_workflow.sh を実行します。WORKFLOW_PROPOSER_BACKEND や WORKFLOW_ROUNDS などの環境変数で挙動を調整できます。

注目ポイント・注意点

LLM が LLM の推論制御アルゴリズムをコードとして設計するという方向性は新しく、得られた結果が人間に読めるプログラムである点が特徴です。探索の評価が安価な点も注目に値します。一方で、注意すべき点もあります。まず、再生環境の構築には事前に大量のトレース生成が必要ですが、その手順やコストの詳細は README からは不明です。次に、評価は数学系ベンチマークと Qwen3 系に限られています。Qwen3-8B では競合水準にとどまったとも記載されています。さらに、ライセンスは明記されておらず、リポジトリには __pycache__ も含まれています。探索を回す場合は Claude Code の LLM 呼び出し費用がかかります。

技術構成: Python 3.12、Conda、numpy、pandas、tqdm、claude-agent-sdk、Claude Code CLI (@anthropic-ai/claude-code)、Node.js (nvm)、OpenRouter API、Codex、huggingface-cli、Qwen3 (バックボーン)
向いている人: テスト時スケーリングや効率的推論（早期停止、Self-Consistency の改良など）を研究する人に向い

ています。LLM エージェントによるアルゴリズム自動発見に関心のある研究者・学生にも適しています。論文結果を再現したい人や、自前の再生環境で制御戦略を探索したいエンジニアにも有用です。評価だけなら軽量な環境で試せます。

テーマ 20. sunshy-1/SpecTTS-Bench

https://github.com/sunshy-1/SpecTTS-Bench

★ 114 ／ フォーク 1 ／ 言語 Python ／ 作成 2025-12-15 ／ MIT
該当テーマ: test-time scaling LLM

LLMのテスト時スケーリングを投機的デコーディングで高速化する手法を公平に比較するICLR'26論文のベンチマーク実装

概要

テスト時スケーリング（Best-of-Nサンプリングや多段階思考など）は推論能力を高める一方で、冗長で繰り返しの多い推論トレースを生成するため計算コストが大きいという課題があります。本リポジトリは、この状況で投機的デコーディングがどれだけ有効かを評価する初の包括的ベンチマークとして、ICLR'26論文のPyTorch実装を公開したものです。model-based、training-based、n-gram-basedの3系統の手法を、統一した実験プロトコルで比較できます。論文では、単純なn-gram系手法が反復パターンをうまく捉え、テスト時スケーリングの高速化に独自の可能性を持つことが示されています。

要点

- 投機的デコーディングをテスト時スケーリング文脈で評価する初のベンチマーク
- EAGLE-3、REST、Lookahead、SAM-Decodingなど9手法を収録
- DeepSeek-R1-Distill-Llama-8BとQwen3系列を対象モデルに対応
- n-gram系手法が反復的な推論の高速化に有効という知見
- Spec-Bench、EAGLE、Medusaのコードをベースに構築、MITライセンス

何をするものか

SpecTTS-Benchは、LLMのテスト時スケーリング（推論時に追加の計算資源を割り当てて推論能力を高める手法）を、投機的デコーディングでどこまで高速化できるかを測るベンチマークです。テスト時スケーリングでは似たような推論過程が何度も生成されるため無駄が多く、この反復の多い構造化された文脈で投機的デコーディングがどう働くかはこれまで十分に調べられていませんでした。本ベンチマークはBest-of-Nサンプリングや多段階思考といった代表的なパラダイムに対して一貫した実験プロトコルを提供し、手法同士を公平に比較できるようにしています。

収録されている投機的デコーディング手法

README によると、対応手法は EAGLE-3、Speculative Sampling、Prompt Lookup Decoding、TokenRecycling、REST、Lookahead Decoding、PIA、SAM-Decoding、SAM[EAGLE-3] です。論文ではこれらを、小さなドラフトモデルを使う model-based、専用のドラフトヘッドなどを学習する training-based、過去の出現パターンから候補を引く n-gram-based の3カテゴリに分けて比較しています。ファイル一覧では code/model 以下に eagle3、lade、pia、pld、recycling、rest、samd、sps といったディレクトリがあり、それぞれの手法の実装が置かれています。

仕組み・アーキテクチャ

コードは code ディレクトリに集約されています。model には各投機的デコーディング手法の実装があり、Llama系（DeepSeek蒸留モデル用の modeling_llama_kv_ds.py など）と Qwen3 用の modeling_qwen3.py が手法ごとに用意されています。evaluation には inference_eagle3.py、inference_rest.py などの手法別推論スクリプトと、ベースラインの inference_baseline.py、評価用の eval.py や speed_cot.py があります。data/SpecTTS_Bench/question_cot.jsonl が推論用データセットで、scripts の deepseek.sh と qwen3.sh から一括実行する構成です。SAM-Decoding 部分には静的・動的なサフィックスオートマトン（static_sam.py、dyn_sam.py）やツリー設定ファイルも含まれます。

技術構成と対象モデル

Python と PyTorch で実装されており、推奨環境は conda で作成する Python 3.10、torch==2.1.1+cu121 です。transformers はモデルによってバージョンが異なり、DeepSeek-R1-Distill-Llama-8B では 4.43.1、Qwen3 系列では 4.53.1 を使います。詳細な依存関係は environment_deepseek.txt と environment_qwen3.txt に記載されています。ターゲットモデルは DeepSeek-R1-Distill-Llama-8B、Qwen3-4B、Qwen3-8B、Qwen3-14B、ドラフトモデルは Qwen3-0.6B と各ターゲットに対応する EAGLE3 の重みで、いずれも Hugging Face からダウンロードします。

使い方・導入方法

まず conda で Python 3.10 環境を作り、対象モデルに合った transformers のバージョンを入れます。次に必要なモデル重みを Hugging Face から取得し、code/scripts 内のスクリプトでチェックポイントのパスを書き換えます。その後 cd code してから bash scripts/deepseek.sh、または bash scripts/qwen3.sh を実行すると評価が始まります。REST 用のデータストアのチェックポイントは README に記載された共有リンクから配布されています。個別の引数や出力形式の詳細は README からは不明なので、スクリプトの中身を確認する必要があります。

注目ポイント・注意点

一番の注目点は、論文の知見として、単純な n-gram 系手法がテスト時スケーリング特有の繰り返しパターンを効率よく捉えることが示されている点です。さらに、n-gram 系を model-based や training-based と組み合わせることで、反復的な推論と多様な推論の両方をバランスよく加速できる可能性が示唆されています。注意点として、DeepSeek と Qwen3 で transformers のバージョンが異なるため環境を分ける必要があります。また、GPU 要件や実行時間、評価指標の具体的な定義は README からは不明です。

技術構成: Python 3.10、PyTorch 2.1.1 (CUDA 12.1)、Hugging Face transformers (4.43.1 / 4.53.1)、conda、EAGLE-3、Spec-Bench、Medusa
向いている人: LLM推論の高速化や投機的デコーディングを研究している研究者・大学院生、テスト時スケーリング（Best-of-N、多段階思考）の計算コストを下げたいエンジニアに向いています。複数の投機的デコーディング手法を同一条件で比較・再現したい人や、新しい手法を評価するための土台を探している人にも有用です。